

FOUNDATION MONITORING RESEARCH

The Research Reference from AASB

Abstract

This document contains all research findings and data collection from the 2025 Foundation monitoring system team, As Above So Below.

Sponsored by

StableOption2



AS ABOVE SO BELOW

Team: Sloan Loudon-Robins, Cassidy Miskovitz, Will Rebenack, Orlando Torres

Written by: Sloan Loudon-Robins, Cassidy Miskovitz, Will Rebenack, Orlando Torres
Latest Version – 12/03/2025

Contents

Overview	0
Overall Block Diagram	0
System Materials	0
Hardware.....	1
Communication Protocol.....	5
Functionality Results	7
Simulation Platforms	8
Platform Functionality Results.....	10
Data Processing	12
Filter Consistency for Python vs C++	13
UI.....	16
UI Functionality Results	19
Sensors	20
IMU.....	20
MPU6050	21
LSM9DS1	22
BNO055.....	24
ToF.....	28
VL53L4CX	28
Differential pressure	30
HSCDRRN010NDAA5	30
PCR Radar.....	30
XM125 – A121 Breakout Board	30
Parameter Settings 0: [Taken from testing log].....	31
Dial Gauge	36
Strain Gauge & Flex Resistor	37
Soil Moisture	39
Sensor Summary (Cost Benefit Analysis).....	40
Filtering and Machine Learning.....	42
Filters.....	42
Machine Learning	43
IMU – BNO055 – 2wk static drift ML model and analysis	44
Configurations	49
Citations.....	50

Overview

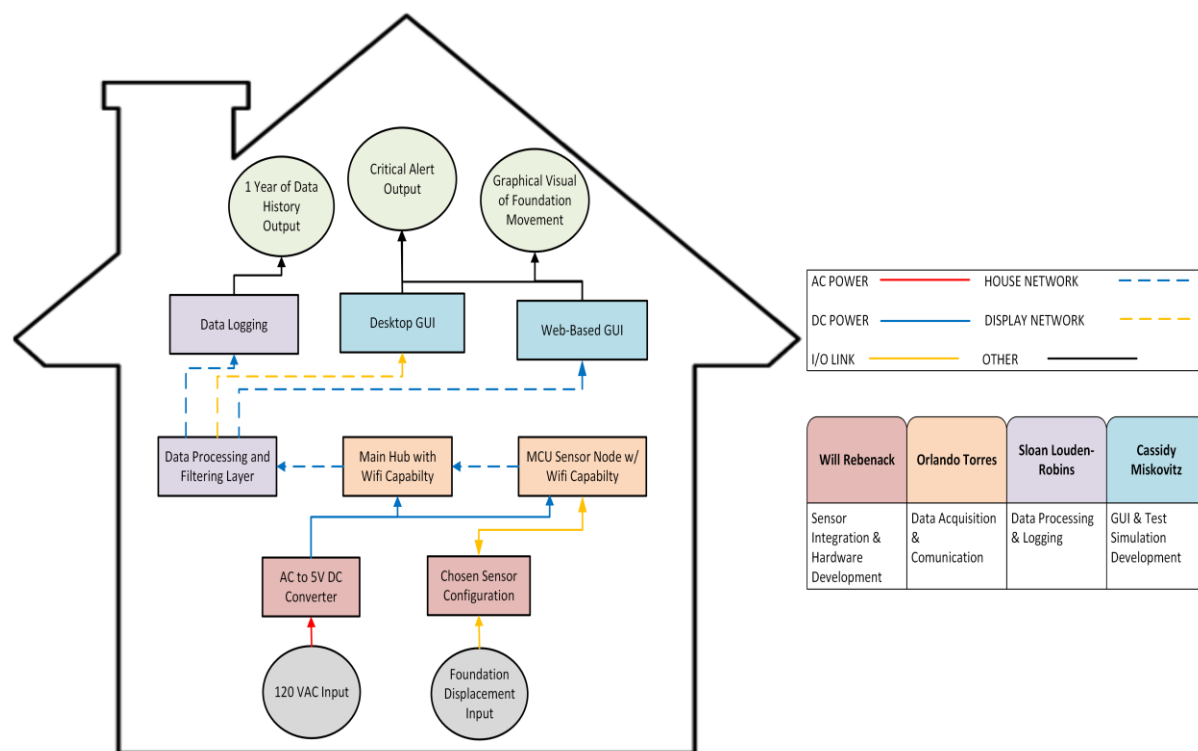
[This section was written by Sloan Louden-Robins]

This Research project began in January 2025 for the spring semester and continued until the end of the Fall semester in December 2025. The team, As Above So Below, researched varying sensors, hardware, communication protocols, user interface implementations, data processing techniques, filtering algorithms, and simulation models with the goal of finding out if there is a way to detect 1mm shifts over a long period of time in a home's foundation. This structural health monitoring system could provide a multitude of functions in varying electrical engineering and STEM fields, as well as saving money for homeowners being able to preemptively detect foundational issues.

This document serves as a reference guide for our findings and for the next team to work on this project.

Overall Block Diagram

[This block diagram was created by Cassidy Miskovitz]



System Materials

MCU:

ESP32-C6 [Datasheet: [ESP32-C6_Series_Datasheet.pdf](#)]

Raspberry Pico [Datasheet: [Raspberry Pi Pico Datasheet](#)]

MPU:

Raspberry Pi 5

Hardware

[This section was written by Will Rebenack]

Overview

My subsystem is responsible for the physical sensing layer of the foundation monitoring system: the wall-mounted sensor nodes, their custom PCBs, and their electrical interfaces to the sensors. Each node consolidates multiple sensors (IMUs, ToF, and PCR Radar) on a compact, ESP32-based board that can be deployed on interior or exterior walls to track long-term tilt and displacement associated with foundation movement. This hardware is the front end that turns physical motion of the house into digital data for the data acquisition, processing, and UI subsystems.

What's been achieved

Selected the BNO055 from a group of IMU candidates carried over from the previous semester and designed a long-term drift analysis to evaluate its suitability against the project's accuracy requirements. The test characterized how the sensor's yaw/pitch/roll output drifted over time under static conditions.

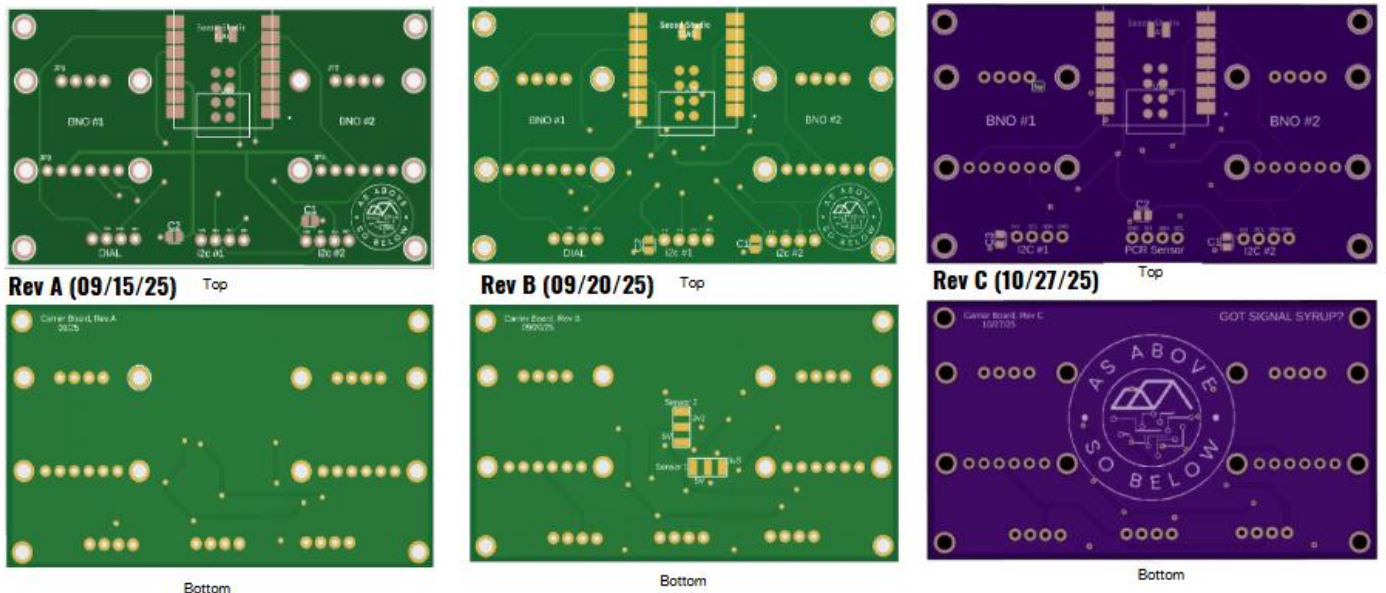
Determined from this drift testing that the BNO055 is not viable as a primary long-term sensing device for this application without frequent re-calibration, although it could still be useful in a secondary role (for example, providing short-term orientation context when combined with other primary displacement sensors).

Designed a custom sensor-node PCB around the Seeed XIAO ESP32-C6, with footprints for two BNO055 IMUs and I²C breakout headers to support additional sensors under evaluation by the team (such as the pulsed coherent radar (PCR) sensor and a Time-of-Flight (ToF) sensor).

Created and iterated through three PCB revisions this semester as the team refined the sensor mix:

- Rev A: Initial board with dual BNO055 support and generic I²C breakouts for experimentation.
- Rev B: Added a jumper on all I²C breakout power pins to toggle the header supply between 3.3 V and 5 V, allowing the team to test a wider range of potential sensors, including any that might require 5 V.
- Rev C: Removed the voltage-select jumpers after confirming that all current sensors operate at 3.3 V, and added a dedicated breakout header specifically for the PCR sensor, simplifying its integration and wiring.

Used these PCB revisions to support teammates' sensor evaluations (PCR and ToF) while keeping the hardware flexible enough to adapt as the sensing strategy evolved.



Key Takeaways

Accurate measurements depend on a solid mechanical reference. Because we're trying to track very small shifts in the house over time, the sensor node must be fixed firmly to the wall so that any change in the readings reflects real structural movement, not motion or sag in the hardware itself.

All sensors drift, but IMU drift shows up differently. Any practical sensor (digital or analog) will exhibit some offset and gain drift over time due to temperature, aging, and internal biases. With IMUs like the BNO055, those small biases get integrated when you compute orientation, which causes the reported angle to "wander" more and more over hours. Our long-term test made that behavior very visible and showed why IMUs are tricky as a primary long-term reference.

Data sheet specs aren't the whole story. The BNO055 looks good on paper, but its real long-term behavior under static conditions didn't match what we needed for slow foundation movement. The drift testing was essential to see how it behaved over hours rather than just short lab runs.

Proving something doesn't work is still progress. Demonstrating that the BNO055 is not a good primary sensor for this application (without constant re-calibration) still moved the project forward. It helped us reposition the IMU as a possible secondary sensor and pushed the team toward displacement-based measurements as the main signal.

Future suggestions

1. Select a primary displacement sensor early

- Test sensor candidates early using controlled displacement setups to evaluate resolution, range, and long-term stability.
- Set the IMU (BNO055) aside during selection; bring it back later only as a secondary/contextual sensor once a reliable primary sensor is confirmed.

2. Plan for outdoor deployment constraints

- Account for environmental factors: temperature changes, moisture, UV exposure, insects, cable strain, and mounting challenges.
- Incorporate these into enclosure design, connector choice, sealing, and sensor placement/orientation.
- Run additional tests to understand how outdoor conditions impact drift and long-term stability compared to indoor tests.

3. Address quantization limits in the measurement chain

- Pay attention to sensor resolution, ADC step size, and digital scaling—small structural movements can get buried in quantization noise.
- Improve visibility of small shifts by selecting optimal sensor ranges, using averaging/filtering, and applying trend-based analysis over longer windows.
- Validate that the full sensing chain can resolve changes at the mm-scale or below, which is critical for this project.

Basic Specs

Power and main interface:

- Sensor-node PCB is powered from a 5 V, 2 A USB-C connector.
- On-board regulation provides a 3.3 V rail for the ESP32-C6 and most external sensors.

Microcontroller / base module:

- Seed XIAO ESP32-C6 used as the main controller.
- Handles I²C communication with all sensors and forwards data to the rest of the system.

I²C breakouts and power domains

- Two generic I²C breakout headers for additional sensors (e.g., ToF).
- One dedicated I²C header for the XM125 PCR module.
- All three headers are powered from 3.3 V and share the ESP32-C6's I²C bus.
- 5 V rail: USB-C input, BNO055 sensors (to avoid loading the 3.3 V regulator in multi-IMU configs).
- 3.3 V rail: ESP32-C6, VL53L4CX, XM125, and other I²C breakout sensors.

PCB Repeatability Results

Experiment Setup:

Sensors

Adafruit BNO055

Mounting

Both boards laid flat on a rigid, non-ferrous surface

Sampling Rate/ Duration

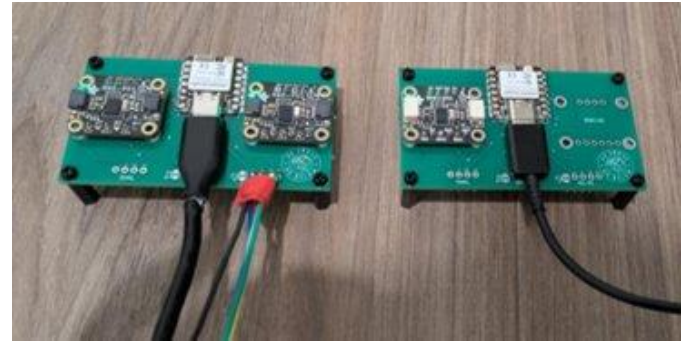
100 Hz (10ms interval) for 60s

Recorded Quantities

Linear acceleration (m/s^2)

Angular velocity (rad/s)

Orientation (Euler angles $^\circ$)



Experiment Setup

All data logged locally to flash memory and later retrieved as CSV files. Each board operated in Nine Degrees of Freedom (NDOF) Mode.

Results Summary

Quantity	Mean Diff	Std. Deviation	Units	Interpretation
accel_x	0.376	0.005	m/s^2	Slight constant offset ($\sim 0.04 \text{ g}$)
accel_y	0.640	0.005	m/s^2	Minor bias ($\sim 0.065 \text{ g}$), highly repeatable
accel_z	-0.143	0.018	m/s^2	Excellent alignment on vertical axis
gyro_x	-0.0003	0.0008	rad/s	Negligible bias and noise
gyro_y	0.0009	0.0015	rad/s	Negligible bias and noise
gyro_z	-0.0003	0.0011	rad/s	Negligible bias and noise
euler_x (Roll)	+0.063	0.000	$^\circ$	Essentially identical roll alignment
euler_y (Pitch)	+1.999	0.000	$^\circ$	Constant offset, likely slight front-back tilt difference
euler_z (Yaw)	-3.625	0.000	$^\circ$	Small constant yaw offset, possibly magnetic or PCB alignment related

Interpretation:

The short-term static repeatability test demonstrated that both BNO055 IMUs produced nearly identical readings under identical conditions, with offsets below 0.07 g and within 4° , consistent with expected manufacturing and mounting tolerances.

This test confirms that the hardware and data pipeline can reliably produce consistent short-term measurements across identical nodes—a critical step before extending testing to long-term drift scenarios.

Communication Protocol

[This section was written by Orlando Torres]

Overview

The MQTT and UDP Auto-Discovery subsystem was developed to create a smooth communication layer for all wireless sensor nodes in the Foundation Monitoring System. This included building the complete workflow for device announcement, broker connection, topic organization, packet-timing validation, and real-time data publishing. The architecture pairs with any MCU and has been shown to work with both the PICO W, and ESP32-C6. Firmware with Python monitoring tools on the Raspberry Pi gateway, allowing each node to automatically advertise its presence, obtain its assigned communication parameters, and begin publishing data with zero manual configuration. Together, the auto-discovery mechanism and the MQTT transport layer form the core of the system's sensor-to-gateway data pipeline.

What's Been Achieved

A fully operational discovery-to-MQTT stack was implemented.

- **UDP Auto-Discovery:** A custom broadcast listener on the Pi responds to chosen MCU discovery packets, assigns a return IP/port, and confirms the node's identity using a simple, collision-resistant message structure.
- **MQTT Publishing Pipeline:** All ESP32-C6 units can automatically connect to the broker after discovery, register themselves using standardized topic naming, and begin streaming JSON data (e.g., IMU, microphone, radar, diagnostics).
- **Python Monitoring Tools:** The Pi hosts Python subscribers for topic visualization, packet-timing checks, and device-level diagnostics.
- **Network Verification:** Wireshark was used extensively to confirm UDP handshake behavior, verify broadcast reachability across subnets, diagnose packet collisions, and validate MQTT publishing timing.

This end-to-end implementation now allows any sensor node to join the network in seconds, self-register on MQTT, and appear immediately in dashboards without manual IP assignment or configuration edits.

Key Takeaways

The reliability of the system is governed by timing across the entire discovery-to-publish pipeline. Small delays in the UDP handshake can cause devices to miss their registration window, while inconsistent publish intervals can destabilize downstream scripts. A second critical lesson is the importance of strict topic standardization—changes to naming conventions or JSON fields propagate to every subscriber, data-logger, and UI stack. Lastly, network-layer validation is essential: debugging MQTT issues often required examining UDP broadcasts, packet loss, or broker-level timing in Wireshark rather than adjusting firmware.

Future Suggestions

- Adaptive discovery retries that adjust the broadcast interval based on network congestion.
- Node-health metadata, such as packet loss percentage, MQTT round-trip time, and uptime counters published to a dedicated status topic.
- A central "Sensor Health Page" on the Pi or Home Assistant that aggregates discovery events, online/offline transitions, and per-topic update rates.
- Long-duration tests (10–30 days) that track timing drift, discovery reliability, and broker uptime under realistic loads.
- Obstructed Testing with walls, metal structures, or enclosure placement to evaluate discovery latency, update rates, and reconnect behavior degrade under real-world signal blockage.

Some Basic Specs

UDP Auto-Discovery Protocol

- Broadcast port: 4210
- Pi responds with broker IP, assigned node port, and acknowledgement token
- Handshake completion: typically <20–40 ms
- Collision handling: randomized back-off in firmware during retries

MQTT Layer

- Broker: Mosquitto on Raspberry Pi
- Topic families: device/<nodeID>/data, device/<nodeID>/status, device/<nodeID>/debug
- Payloads: structured JSON (raw + filtered fields depending on sensor)
- Update rate: ~20–100 Hz depending on node configuration

Python Monitoring & Tools

- Auto-subscriber for any active node
- Reconnect logic with adjustable timing
- Real-time terminal + plotting utilities
- Logging for packet timing, drift, and missed frames

Network Testing

- Wireshark for UDP broadcast reachability, timing jitter, collision inspection
- MQTT client testing via mosquitto_pub/sub
- End-to-end acquisition loop latency typically 5–15 ms

Functionality Results

Procedure	Expected Result	Actual Result	Comments:
Start Mosquito Broker	Broker accepts TCP connections on port 1883	Broker Listening Confirmed	Passed.
Connect 6 virtual clients using mqtt_multi_pub.py (QoS 1, 10 Hz, 60 s).	All 6 clients publish 10 Hz each (total $\approx$ 3600 msgs)	6 clients connected and completed without disconnects.	Passed.
Capture MQTT traffic with Wireshark/tshark during test.	.pcapng file includes PUBLISH and PUBACK for all topics.	Capture successful.	Passed: Tshark used will verify with Wireshark
Extract timestamps for PUBLISH and PUBACK using tshark filters	Two CSVs (publish.csv, puback.csv) created.	Completed successfully.	Passed : 3598 of 3600 messages matched.
Analyze data in Python using analyze_mqtt_latency.py.	Mean $\leq$ 40 ms, p95 $\leq$ 80 ms, PDR $\geq$ 99 %.	Mean = 33.7 ms, p50 = 28.1 ms, p95 = 77.2 ms, p99 = 89.6 ms, σ = 10.8 ms, PDR = 99.94 %.	All metrics within limits, but might be improvable

Metric	Description / Measurement Criteria	Units	Result
Connection Time	Time from power-on to successful MQTT broker connection	s	~2.5
Auto-Reconnect Time	Time from disconnection to successful automatic reconnection(NOT live display)	s	~2.5
Active Devices	Number of active sensor nodes tested	count	6
Active Users	Number of clients subscribed/viewing live data	count	4
Average Latency	Mean delay between data capture and broker receipt	ms	32
Packet Loss Rate	Fraction of messages lost or dropped during transmission	%	<2
Throughput	Average data rate per device (payload $\times$ publish frequency)	kB/s	14.2
System Uptime / Stability	Longest continuous runtime without manual reset	h	435

Simulation Platforms

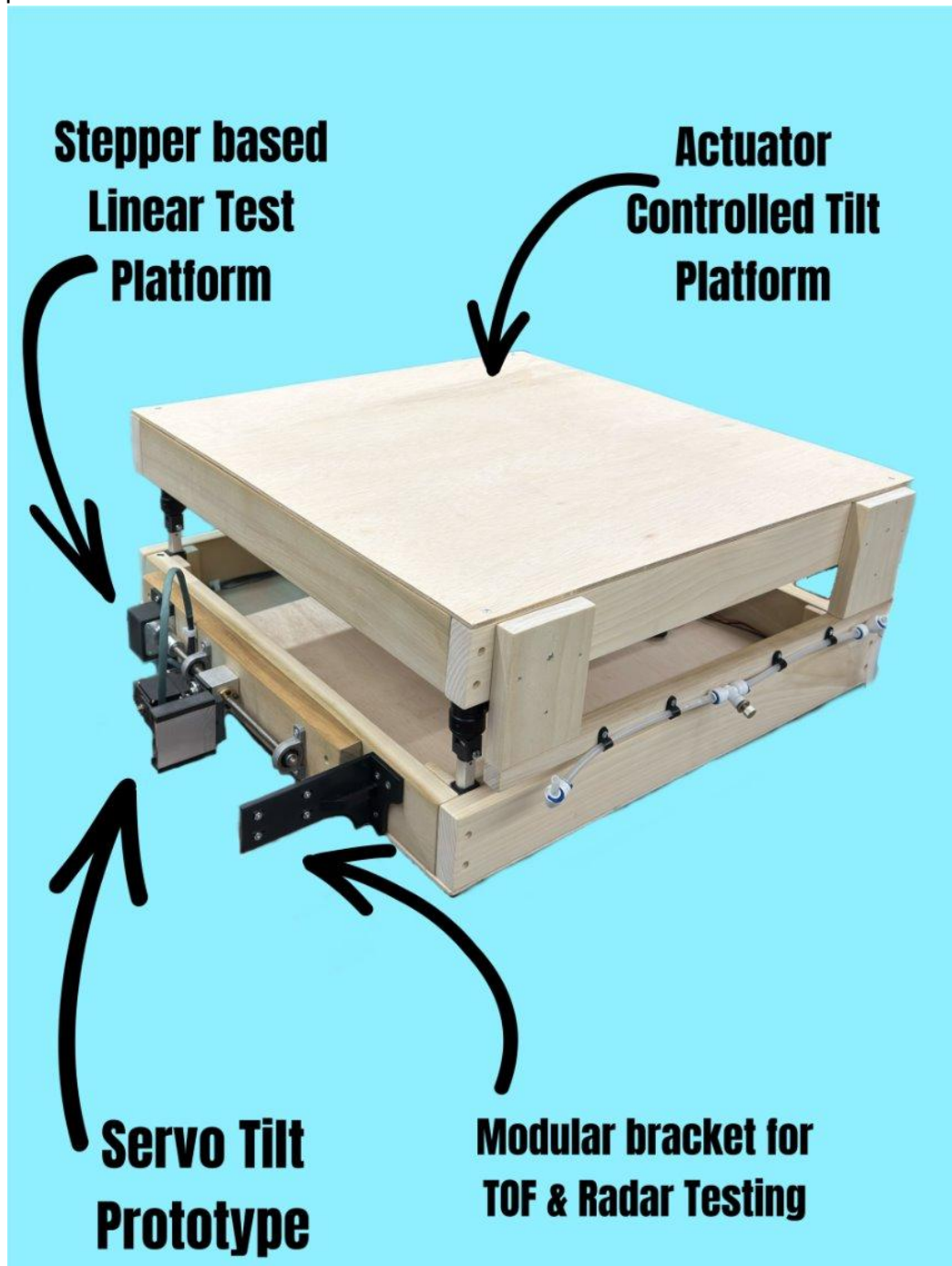
[This section was written by Cassidy Miskovitz]

Overview

The test rigs were developed to verify that each sensing subsystem; ToF, IMU, radar, and liquid level performed as expected before being deployed in a home. They allowed us to simulate controlled movement, tilt, and drift so we could confirm accuracy and precision

What's Been Achieved

We built several mechanical testing platforms, including a linear stepper rig, a servo-driven tilt rig, and a four-actuator tilt simulator. A liquid-level test column with a ToF verification setup was also completed. These rigs produced repeatable data that helped validate filtering, calibration, and long-term sensor performance.



Key Takeaways

Testing in the real world mattered far more than relying on theory or simulation. The rigs exposed noise, drift, and mechanical challenges that we would not have caught otherwise. They also helped the team standardize data formats and test procedures. Long-duration testing turned out to be essential.

Future Suggestions

Future teams should automate the rigs with repeatable motion patterns to support machine-learning calibration and more consistent testing. Adding environmental factors like temperature and humidity would improve realism. A compact “sensor diagnostic box” would also be useful for quick validation during installation or maintenance.

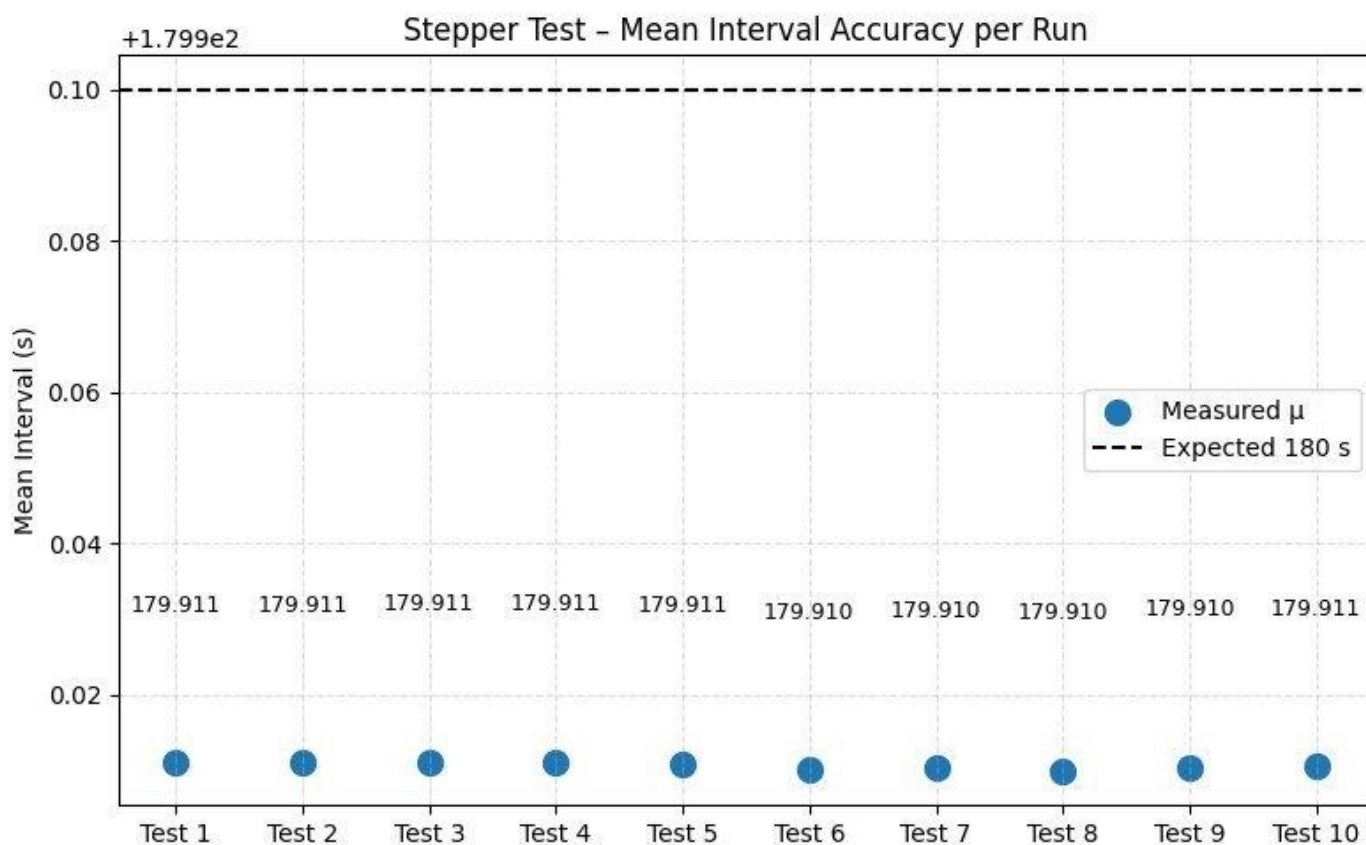
Some Basic Specs

- **Linear stepper rig:** This rig provided precise linear motion for controlled distance testing, allowing us to evaluate accuracy and repeatability across several sensors.
- **MG996R servo tilt platform :** The servo-based tilt rig produced programmable angle changes used to analyze radar response and filtering behavior.
- **Four-actuator tilt platform:** This rig provided rapid, controlled motion for demonstration purposes and was used specifically to showcase liquid-level responsiveness and IMU behavior under dynamic tilt.
- **Water-level column with float and ToF measurement :** This setup simulated hydrostatic behavior and verified that the ToF sensor could accurately track surface level changes through a stable liquid column.

Platform Functionality Results

Stepper Motor:

Stepper Platform Tests: 10 one-hour tests at 1 cm per run to verify repeatability



Parameter	Expected	Actual	Difference	Percent Error
Average Mean Interval (s)	180	179.91064	0.089360	0.049644%
Intervals per hour (N)	20	20.009934	0.009934	0.049670%
Expected Distance (mm)	10	10.004967	+0.004967	0.049669%

- All percent errors < 0.05%
- Timing deviation only 0.089 s from target
- Distance error +0.005 mm
- Intervals per hour within 0.01 of expected

Overall, as a testing and simulation platform it is a **PASS**.

Linear Actuator

Linear Actuator Test: determine if the actuator could move 1 cm in 1 hour continuously and reliably.

Calculations:

$$\begin{aligned} \text{Full-speed (100\% PWM)} &= 1 \text{ cm in } 30 \text{ s} = 0.333 \text{ mm/s} \\ 2 \text{ cm/min} &= 0.333 \text{ mm/s} \\ \text{Target} &= 1 \text{ cm/hour} = 0.00278 \text{ mm/s} \\ \text{If it scaled perfectly linearly} \\ \text{PWMneeded} &= 100\% \times 0.0028 / 0.833 \approx 0.83\% \end{aligned}$$

PWM Duty (%)	Average Speed (cm/min)	Motion Behavior	Notes
10 %	0.0	No motion	No measurable response
20 %	0.0	No motion	Stalled completely
25 %	0.00	No Motion	Stalled completely
30 %	0.29	Moves intermittently	Borderline stall
40 %	0.60	Uneven but steady	Marginal torque
60 %	1.13	Slight jitter	Stable overall
80 %	1.60	Smooth, steady	Reliable operation
100 %	2.00	Smooth, full-speed motion	Normal operation

- The actuator cannot operate continuously below the 30 % PWM threshold (need 0.8% PWM)
- Incapable of sustained motion slow enough to achieve 1 cm/hour.

Overall result is a PASS for simulations and FAIL for testing.

Data Processing

[This section was written by Sloan Loudon-Robins]

Overview

The Data Processing subsystem was developed to create a reliable and scalable workflow for analyzing the sensor data collected throughout the project. Using an Anaconda environment and modular Jupyter notebooks, I built pipelines capable of handling IMU, ToF, radar, soil moisture, strain gauge, flex resistor, and differential pressure data.

Each notebook serves as an experimental workspace for filtering, interpolation, timestamp alignment, and feature extraction. The system is intentionally flexible—any new sensor can be added to the same framework with minimal configuration. While the mechanical dial gauge was evaluated early on, it required little filtering and was de-prioritized due to time and cost constraints.

What's Been Achieved

- Developed multiple sensor-specific Jupyter notebooks for ingestion, cleaning, filtering, and export.
- Verified real-time IMU and ToF logging via Raspberry Pi + MQTT pipelines.
- Implemented a full suite of noise-reduction tools, including Hampel, EMA, Butterworth, Madgwick, Kalman, and custom “Signal Syrup” filters.
- Created timestamp-based synchronization routines for IMU, ToF, and radar data.
- Completed feature correlation and static/shift testing for the A121 radar system.
- Performed notebook validation for all current sensors to ensure consistency and reliability.
- Built early ML components, including weighted vector scoring and drift-trend modeling for long-term IMU datasets.
- Integrated the LEAP2 compute cluster for large-set processing, significantly improving analysis time.

Key Takeaways

- Filtering and consistency checks are more important than early ML complexity; simple methods often produced the highest reliability.
- Every sensor behaves differently—no single filtering strategy works across all types, reinforcing the need for modular pipelines.
- Timestamp alignment is critical for multimodal analysis and cross-sensor correlation.
- Radar processing requires significant preparation; reflector design, sweep configuration, and environmental factors heavily influence results.
- The subsystem stayed in “yellow” due to the volume of incoming data, emphasizing that data management—not hardware—was the primary bottleneck.

Future Suggestions

- Package the notebook functions into a unified Python library for automated nightly data ingestion and filtering.
- Expand ML integration (V1.0 → V2.0) to support anomaly classification, drift prediction, and threshold auto-tuning.
- Introduce standardized metadata for sensor configurations to improve reproducibility in future semesters.
- Conduct long-duration testing for radar and ToF similar to the month-long IMU drift studies.
- Implement automated validation scripts to test filtering behavior whenever new sensors are added.

Some Basic Specs

Software & Tools

- Anaconda/Jupyter ecosystem
- Python (NumPy, Pandas, SciPy, scikit-learn, Matplotlib, stats models)
- LEAP2 HPC cluster for large-dataset processing

Filtering Methods

- Hampel outlier rejection
- Exponential Moving Average (variable half-life)
- Butterworth low-pass
- Windowed median filters
- Madgwick orientation, Kalman-style smoothing

Supported Sensors

IMU, ToF, radar (A121), strain gauge, flex, soil moisture, differential pressure, dial gauge (minimal filtering)

Outputs

Cleaned CSV/HDF5 files, drift-trend models, correlation matrices, threshold-based anomaly flags, Grafana visualizations.

Filter Consistency for Python vs C++

XM125

Run 1

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.999	112.030
Hampel→EMA	NaN	111.888

Run 2

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.945	67.814
Hampel→EMA	0.910	111.935

Run 3

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.898	87.771
Hampel→EMA	0.888	20.920

Run 4

Comparison	Correlation (R)	RMSE (mm)
Hampel	NaN	105.887
Hampel→EMA	NaN	105.887

Run 5

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.958	82.403
Hampel→EMA	-0.080	82.917

Run	Result	Notes
1	FAIL (scale)/ PASS (shape)	0 timing issues 5min @ 111.5mm Excellent correlation but large RMSE due to unit mismatch ($\times 1000$). Functionally correct, numerically off.
2	FAIL	1 timing issue Start @ 111.5mm DECREASING in distance until total distance 21.2mm Algorithmic match good, but EMA parameter mismatch caused drift.
3	PASS	7 timing issues START @ 21.2mm INCREASING in distance until total distance EMA works very well here—low RMSE; Hampel slightly under 0.9 correlation.
4	FAIL	41 timing issues START @ 111.4mm STATIC with +7 offset in code Python data constant; no valid comparison possible. Logging/export failure.
5	PASS (Hampel) / FAIL (EMA)	104 timing issues Jitter movement @ 60mm-140mm Hampel consistent; Python EMA output flat ($\alpha \approx 0$). Implementation issue in Python, not Arduino.

VL53L4CX

Run 1

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.853	0.285
Hampel→EMA	0.853	0.284

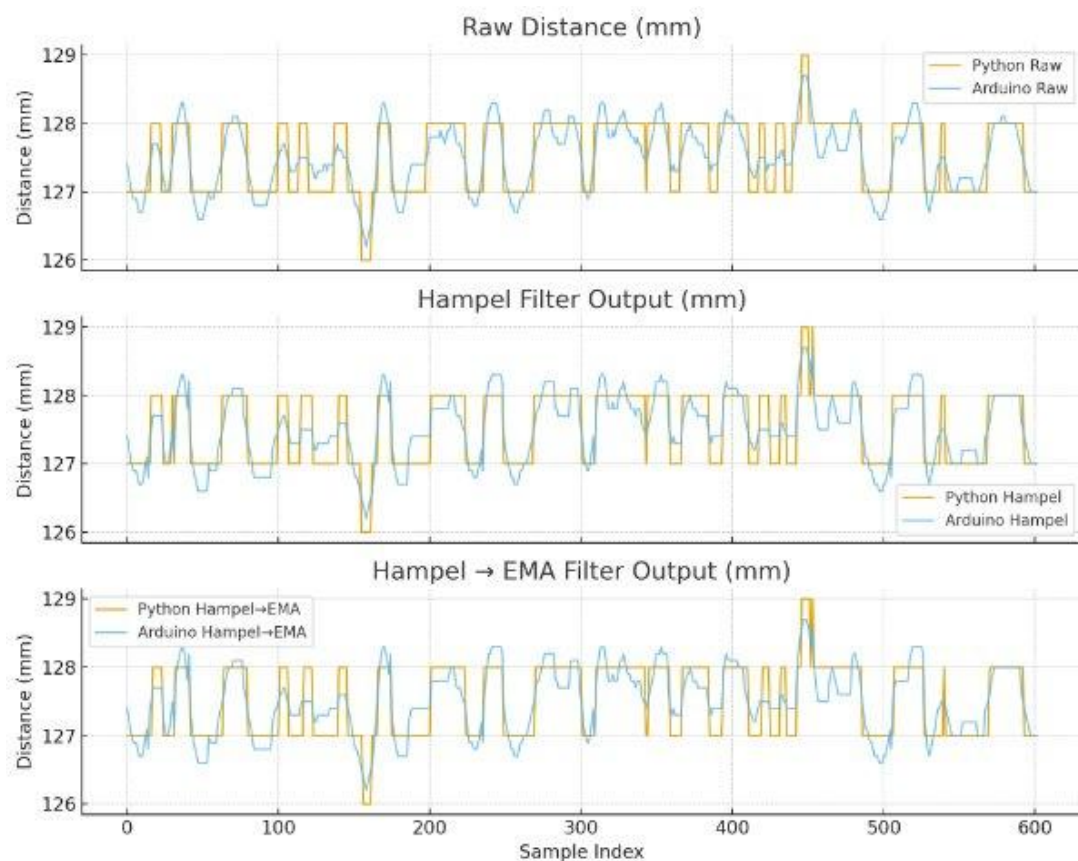
Run 2

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.995	0.283
Hampel→EMA	0.995	0.282

Run 3

Comparison	Correlation (R)	RMSE (mm)
Hampel	0.993	0.321
Hampel→EMA	0.993	0.320

Run	Result	Notes
1	PASS	0 timing issues Static Test
2	PASS	0 timing issues 1cm in 5min -> forwards
3	PASS	0 timing issues 1cm in 5min -> backwards



UI

[This Section was written by Cassidy Miskovitz]

Overview

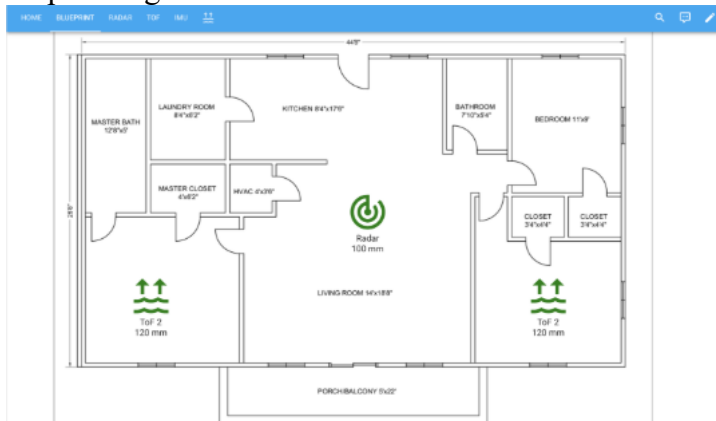
The User Interface for the Foundation Monitoring System was designed to make complex structural-movement data understandable for any homeowner while still providing the depth needed for engineers and future student teams. Built on Home Assistant running in Docker, the UI acts as the centralized hub for all processed sensor data coming from the Raspberry Pi gateway. It provides movement updates, historical insight, visual clarity, and a stable environment that can operate offline through a dedicated hotspot. The entire system emphasizes reliability, low-maintenance operation, and long-term data visibility.

What's Been Achieved

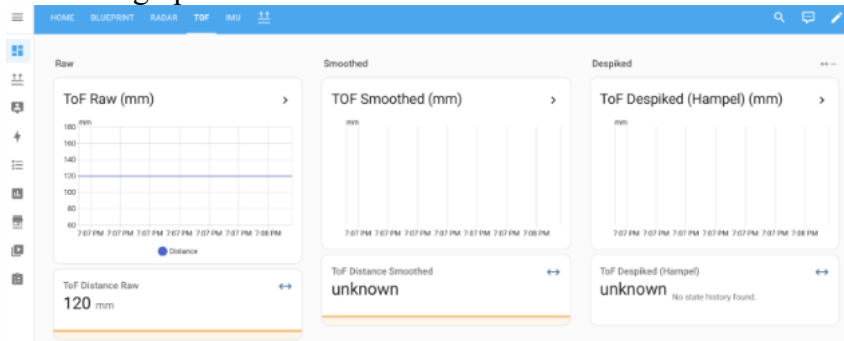
We created a fully functional, multi-page interface that displays foundation movement across several sensing subsystems: ToF, IMU, radar, and the liquid-level module. The UI includes a Blueprint Page for at-a-glance health, Sensor Detail Pages with raw and filtered values, and a Liquid Level Page with custom visualization elements. All data flows through MQTT topics into Home Assistant, which updates daily and stores long-term trends. Custom cards, push notifications, graphs, thresholds, icons, and color-coded warnings make the interface both technical and intuitive.

Each page has a specific purpose:

- **System Status Page:** a top-level glance at whether the house is stable or showing warning trends.



- **Sensor Detail Pages:** breakdowns for ToF, IMU, and water level modules, each with raw values, filtered data, and historical graphs.



- **Liquid Level Page:** shows real-time height changes from the hydrostatic sensor, with a simple visual column and filtered values to highlight subtle movement.



This structure keeps the UI easy for homeowners, while still powerful for anyone installing or maintaining the system.

Key Takeaways

The most important lesson was balancing simplicity with capability. Home Assistant provided structure, but full customization required deeper work such as HTML iframes, custom cards, CSS tweaks, and MQTT tuning. The interface is only as strong as the data pipeline feeding it, so consistent message formatting and filtering are essential.

Future Suggestions

Future teams should strongly consider implementing user account management, which would allow role-based access control. This means separating homeowner views from engineering diagnostics. Along with that, a dedicated Sensor Health Check page would be extremely valuable, showing real-time connectivity, last message timestamps, and whether each module is operating within normal ranges. This single snapshot would eliminate guesswork during troubleshooting and make site visits faster and more efficient.

Finally, while Home Assistant served as an excellent rapid-development platform, future teams may benefit from building a standalone custom UI that does not rely on HA. Using the concepts developed in this project teams could create a lightweight web interface tailored specifically to structural monitoring. This approach would reduce dependencies, increase customization flexibility, and allow the system to evolve beyond HA's limitations while keeping the same design structure established in this version.

Some Basic Specs

- Home Assistant in Docker on Raspberry Pi OS Lite:

Running Home Assistant in Docker allows the UI to remain modular and easily maintainable without relying on the limitations of Home Assistant OS. Using Raspberry Pi OS Lite gives full system control, enabling custom networking, local MQTT brokers, and direct file access for UI pages.

- **MQTT real-time updates:**

Sensor data is ingested in real-time MQTT messages that reflect movement or drift as it happens. This update model ensures that the system remains responsive while still preserving consistent long-term logs for analysis.

- **Local-only hotspot option (wlan1):**

The Pi is configured with a dedicated Wi-Fi dongle on wlan1 that broadcasts a private network when the system is installed in a home without internet access. This guarantees that all sensors can still publish data to the gateway and UI, keeping the system fully functional offline.

- **Custom cards, iframes, and CSS-styled elements**

The UI uses custom Home Assistant cards, embedded iframes, and direct HTML/CSS styling to display complex sensor data in a more technical, engineered format than the default HA interface. These components allow features like 3D liquid level visuals, status highlights, and colored thresholds that match the design language of an industrial monitoring system.

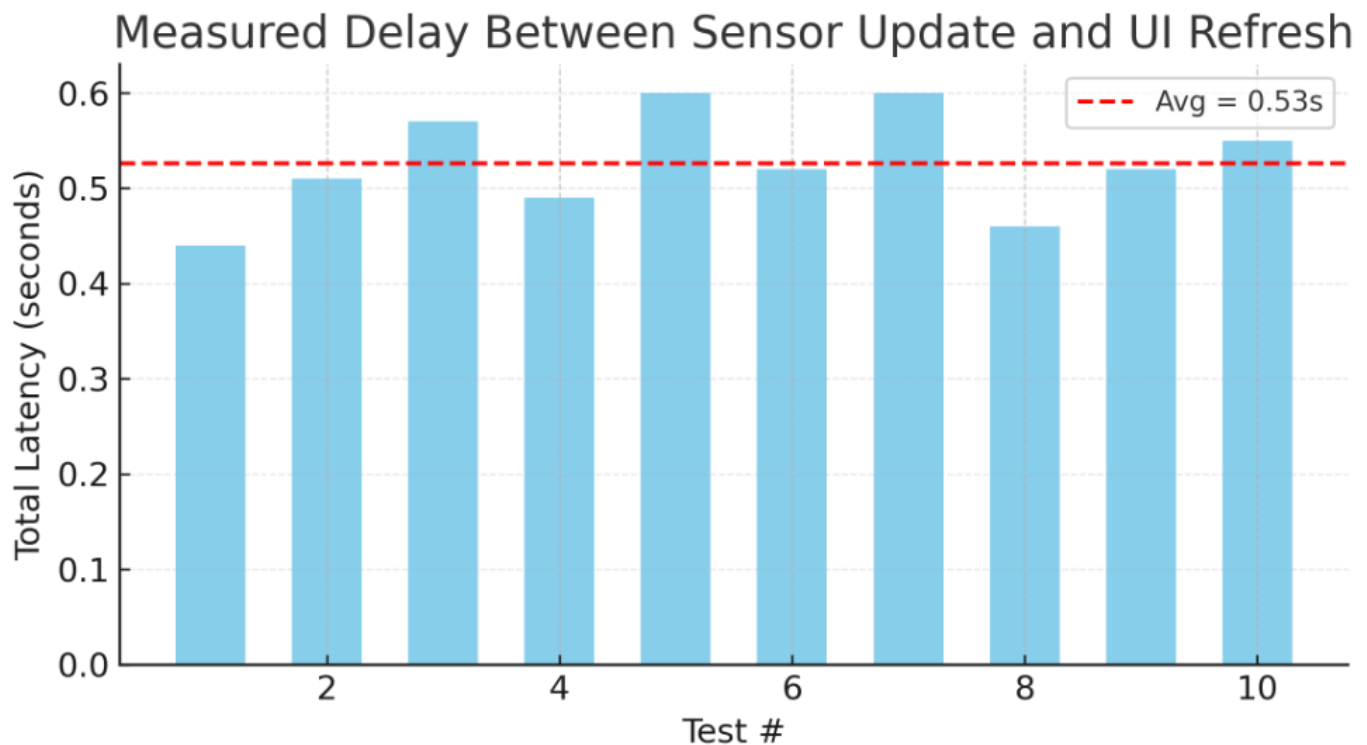
- **JSON-formatted MQTT payloads with filtered and raw data**

Each sensor publishes both raw measurements and noise-reduced values in structured JSON, enabling the UI to display clean, meaningful data while still preserving access to the original readings for debugging. This format also makes the system more expandable, allowing future teams to add new parameters or sensors without breaking the existing UI.

UI Functionality Results

Test 1 – Update Speed:

This test evaluated the delay between each sensor update and the corresponding UI refresh. Across ten trials, the system consistently operated within real-time performance requirements, with an average latency of approximately **0.53 seconds**. All samples remained below the **1-second** threshold, and the variation between trials was minimal ($\sigma \approx 0.05$ s), demonstrating stable and predictable refresh behavior. Overall, the update speed met specifications and the test result is **PASS**.



Test 2 – Node Verification:

This test confirmed that each sensor node was correctly mapped to its corresponding UI label and dashboard page. All nodes displayed accurate identification on the main interface, and every button edirected reliably to the proper detailed sensor page. Navigation remained instantaneous even under active MQTT traffic, indicating that UI routing is robust and unaffected by data load. All requirements were met, and the test result is **PASS**.

Sensors

[This section was written by Sloan Louden-Robins]

Overview

Selecting and evaluating the right sensors was one of the most critical components of the foundation monitoring project. Because the system aims to detect structural movement at the millimeter scale over long periods of time, each candidate sensor had to be assessed not only for accuracy and precision, but also for stability, drift behavior, environmental robustness, and overall practicality for real homes. No single sensor class is inherently suited to every type of displacement measurement, so this stage of the research focused on understanding how each technology behaves under realistic conditions, where factors like noise, temperature, mounting, and time-based drift significantly affect performance.

Across the semester, the team explored a wide range of sensing modalities—including IMUs, optical Time-of-Flight sensors, pulsed coherent radar, strain- and flex-based sensors, differential pressure measurement, soil moisture probes, and metrology-grade digital dial gauges. Each sensor type offers different advantages: some excel at measuring tilt, others at direct displacement, and some serve mainly as contextual or environmental indicators. Just as importantly, each sensor presents unique limitations that impact whether it can be deployed in a long-term structural-health system.

The goal of this section is to document how each sensor performed in controlled testing, highlight the trade-offs associated with its use, and identify which technologies show the most promise for long-term integration. This analysis ultimately guided the project toward the two most viable displacement-measurement approaches—Time-of-Flight and pulsed coherent radar—while also clarifying the supporting roles of secondary or contextual sensors. Together, these evaluations create a comprehensive understanding of the sensing landscape and inform the next team's decisions as the system evolves.

IMU

[This section was written by Sloan Louden-Robins]

An IMU is a multi-axis sensor package that measures motion through accelerometers, gyroscopes, and sometimes magnetometers. It provides information such as linear acceleration, angular velocity, orientation, and drift over time. IMUs are widely used in robotics, drones, automotive systems, mobile devices, biomechanics, and any application requiring motion tracking or stabilization. Because IMUs offer continuous temporal data, they are excellent for detecting subtle shifts or orientation changes even when no external reference is available. However, they naturally accumulate drift, making filtering and long-term correction essential for high-accuracy applications. [1][2][3]

RESULT: FAIL

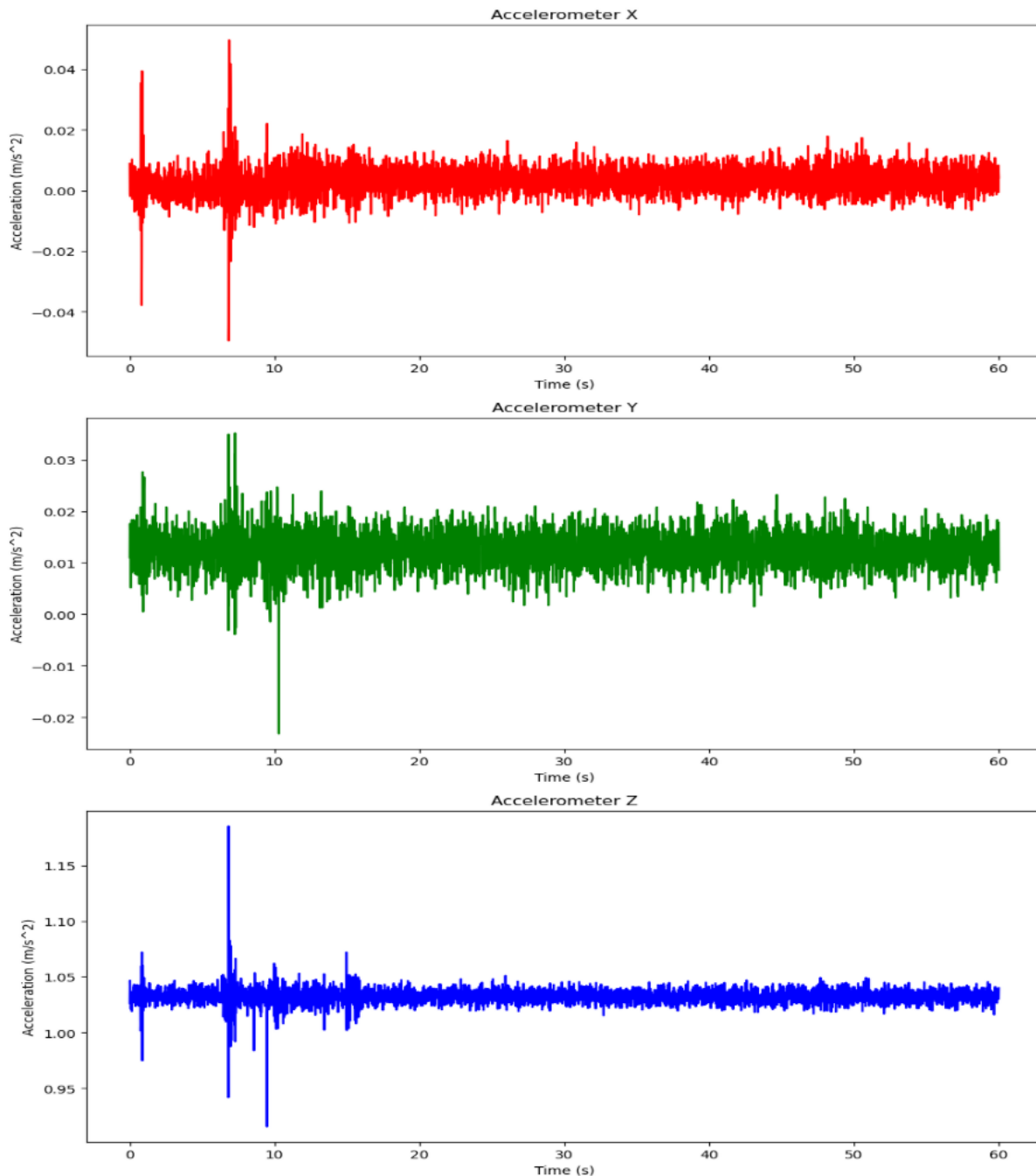
Overall, the IMUs experienced too much drift and are not applicable as a main source for detecting foundation movement to the 1mm scale. However, it could be used as a secondary sensor but there are some other considerations with this. The biggest concerns for a secondary usage of this sensor are the sensor's lifetime, calibration frequency and computational load. For these reasons it fails at being a sensor to explore in depth for the overall system.

MPU6050

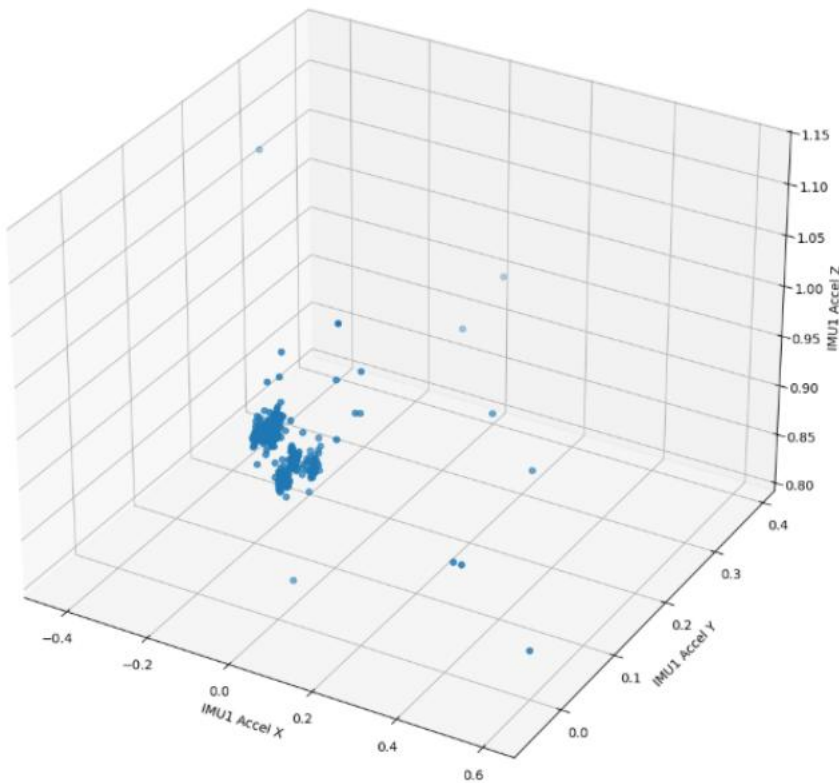
[This section was written by Sloan Loudon-Robins]

Datasheet: [MPU-6000-Datasheet1.pdf](#)

The MPU6050 is a 6-axis inertial measurement unit combining a 3-axis accelerometer and 3-axis gyroscope on a single chip. It provides raw motion data used for acceleration, rotation rate, and basic orientation estimation. Because it lacks a magnetometer and onboard fusion algorithm, it requires external filtering (e.g., Kalman, Madgwick) to stabilize drift. It's widely used in drones, robotics, and embedded projects due to its low cost and fast sampling rates.



Scatterplot of raw MPU accelerometer data

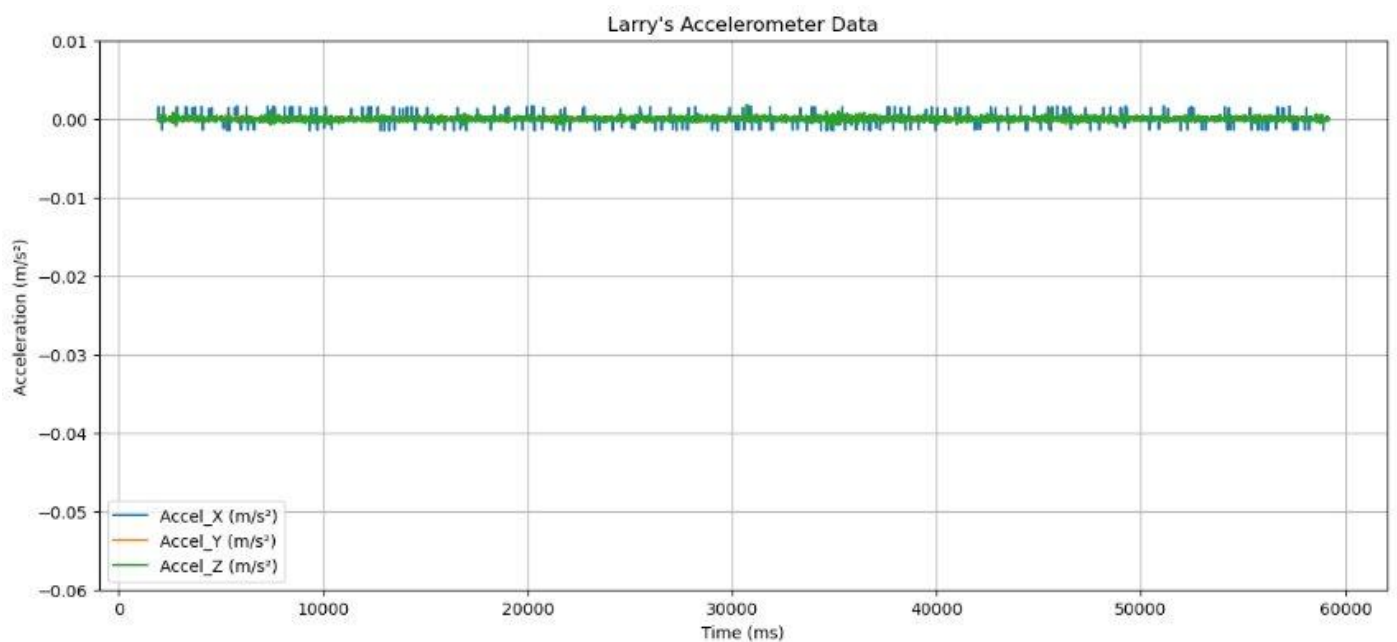


LSM9DS1

[This section was written by Sloan Loudon-Robins]

Datasheet: [LSM9DS1 by STMicroelectronics Datasheet | DigiKey](#)

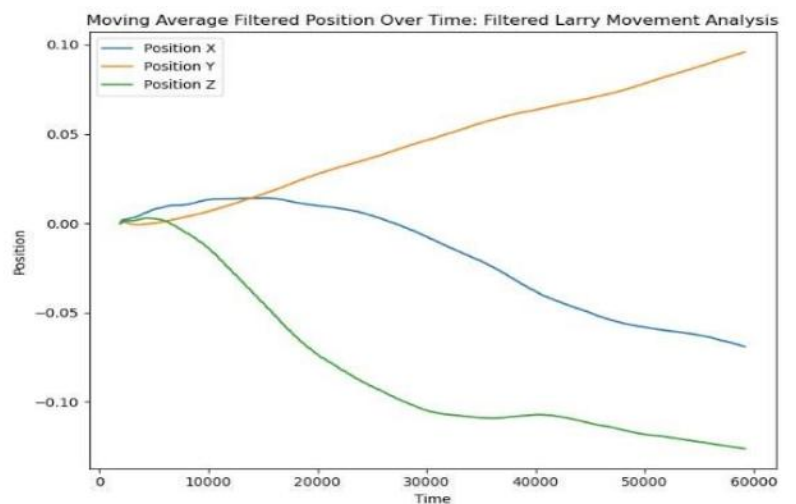
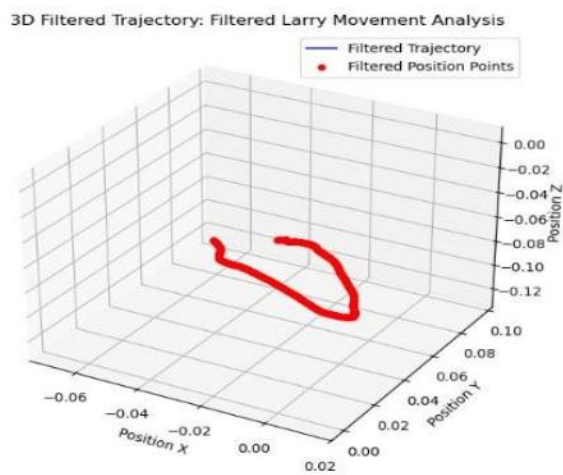
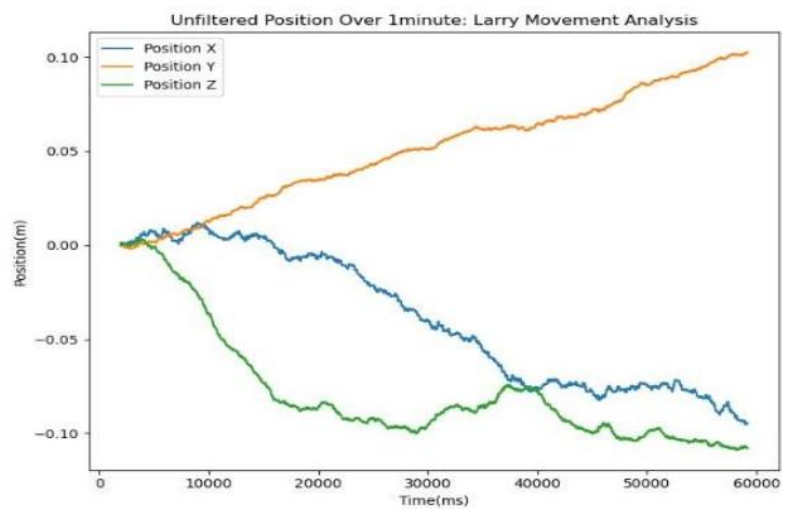
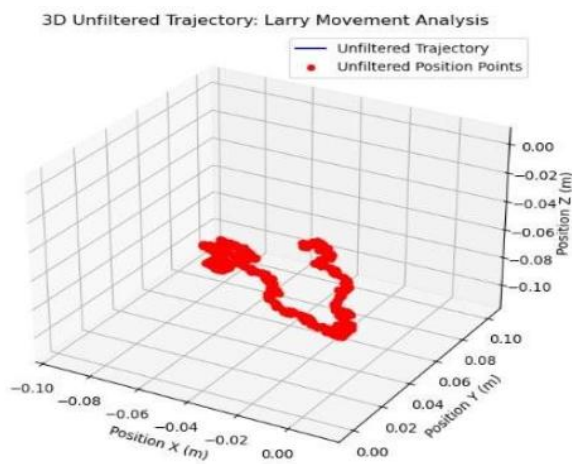
The LSM9DS1 is a 9-degree-of-freedom IMU that integrates a 3-axis accelerometer, 3-axis gyroscope, and 3-axis magnetometer. Unlike the MPU6050, it supports full motion sensing with heading estimation. It's commonly used in robotics, AR/VR, and inertial navigation where a fusion-ready 9-axis package is needed. Although it provides more sensing modes, orientation accuracy still depends on external filtering and calibration.



Filtered and Unfiltered

[This section was written by Sloan Loudon-Robins]

The figures below demonstrate the expected output for a LSM9DS1 movement filtered and unfiltered analysis. IMU was placed static for 1 minute, helping understand how quickly displacement error due to drift begins happening.



BNO055

[This section was written by Sloan Louden-Robins]

Datasheet: [BNO055 Datasheet](#)

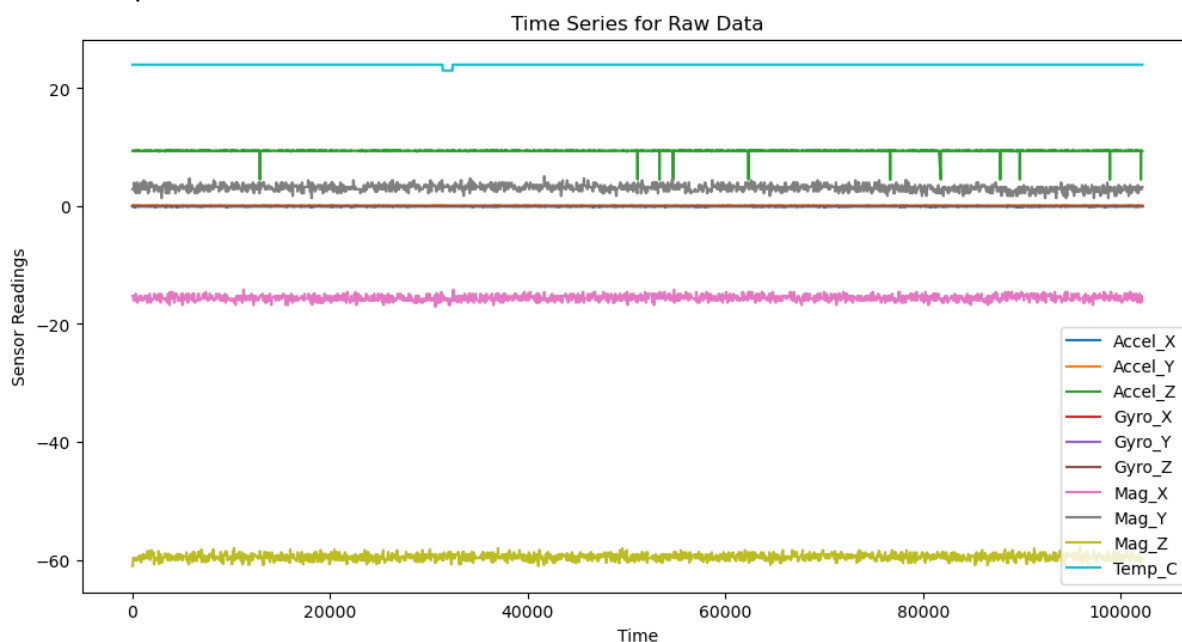
The BNO055 is a 9-axis IMU that includes an accelerometer, gyroscope, and magnetometer, but its defining feature is the built-in sensor-fusion processor that outputs stable orientation (quaternions, Euler angles) directly. This makes it ideal for applications needing drift-reduced real-time orientation without heavy filtering on the microcontroller. It is widely used in robotics, motion-tracking, and embedded control systems where ease of integration and stable heading are important.

Raw vs Calibrated:

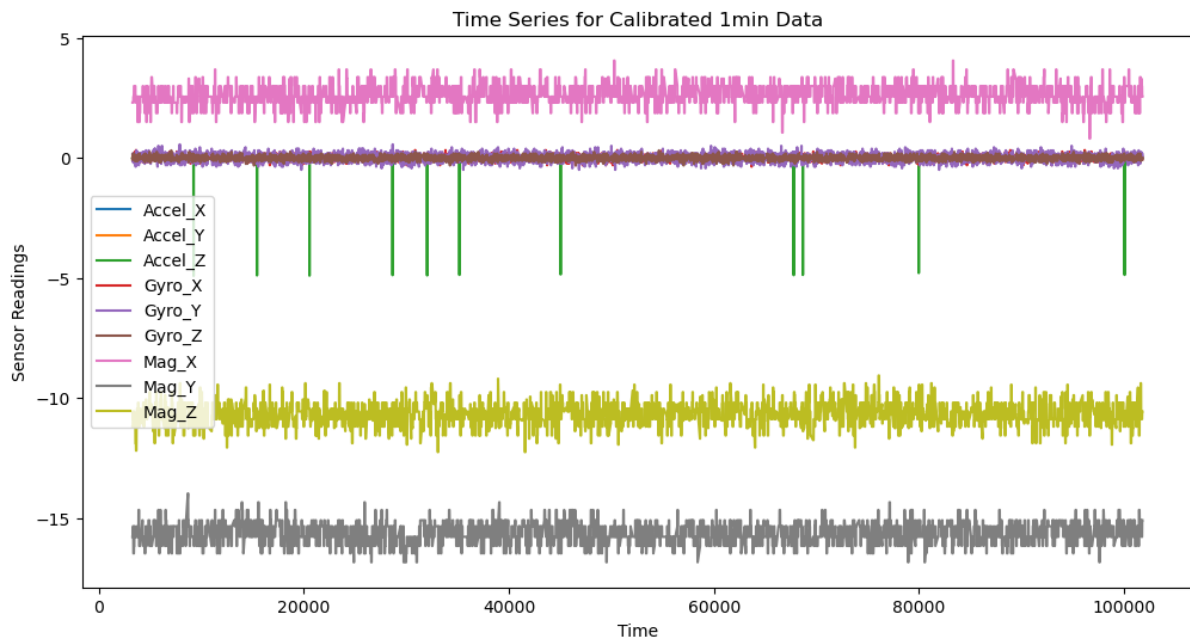
[This section was written by Sloan Louden-Robins]

Below are two time series plots showcasing the difference between the calibrated and raw data output from the BNO sensor. Each plot was evaluated at around a minute, and the Y axis is unitless as each measurement – Accelerometer, Gyroscope and Magnetometer measurements are different in units. Because the BNO was placed static for one minute while collecting data the expectation is the plots should also be at 0. Note that the raw data is very unreliable and not accurate or precise. This is the expectation knowing that the IMU has not been calibrated for these readings. Overall, the performance improved immensely after calibration, however, note that the magnetometer Y axis measurement became further away from the target of being at zero after calibration. This is likely due to environmental magnetic interference from nearby metallic objects, power sources, or any electronic devices. Another interesting observation is that the magnetometer Z axis reading remains inconsistent in both plots suggesting the possibility of localized magnetic distortion or sensor axis misalignment. Overall, calibration effectively reduced measurement noise and systematic bias in most axes. The residual anomalies from the magnetometer axes warrant further investigation if this sensor was to be selected for the overall system.

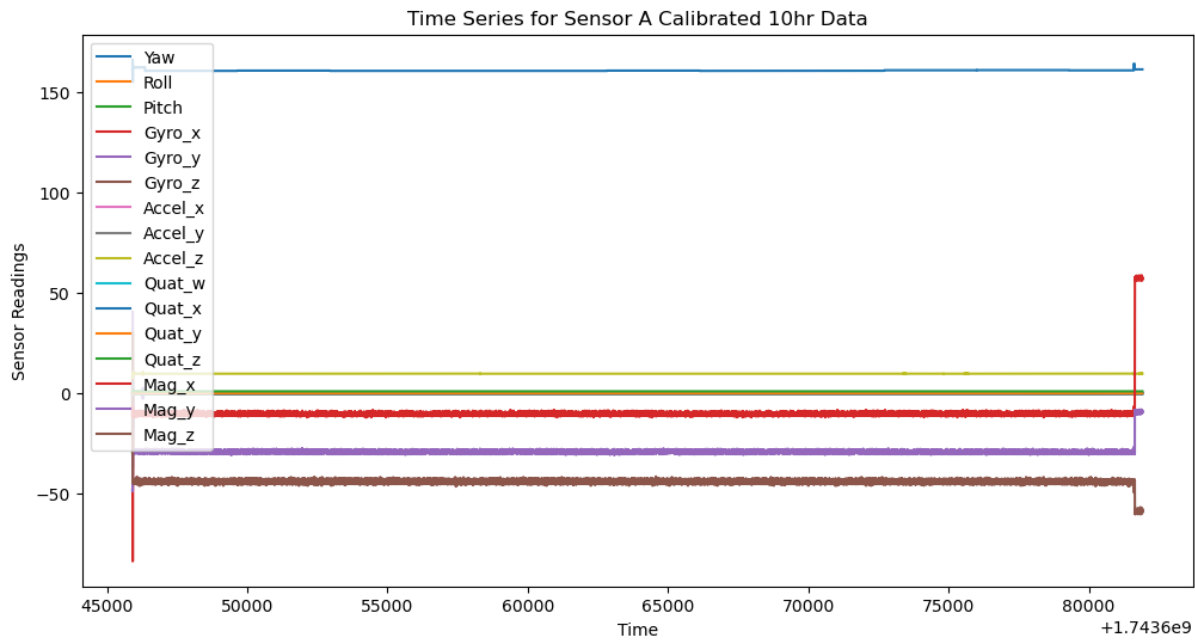
Raw data plot:



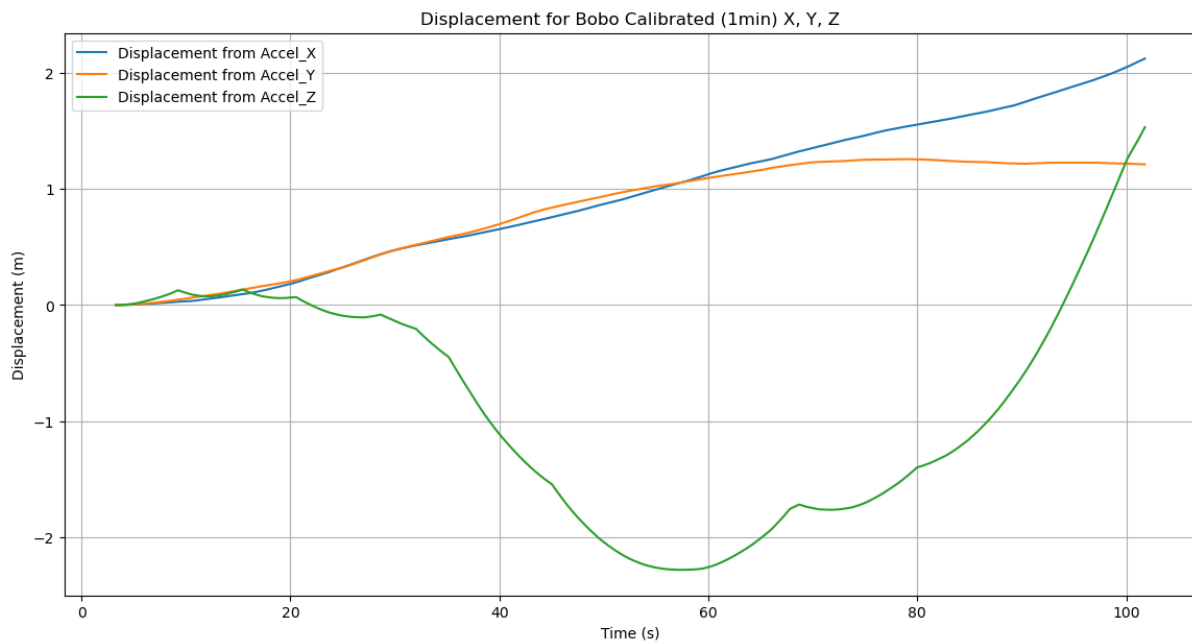
1 minute of Calibrated data



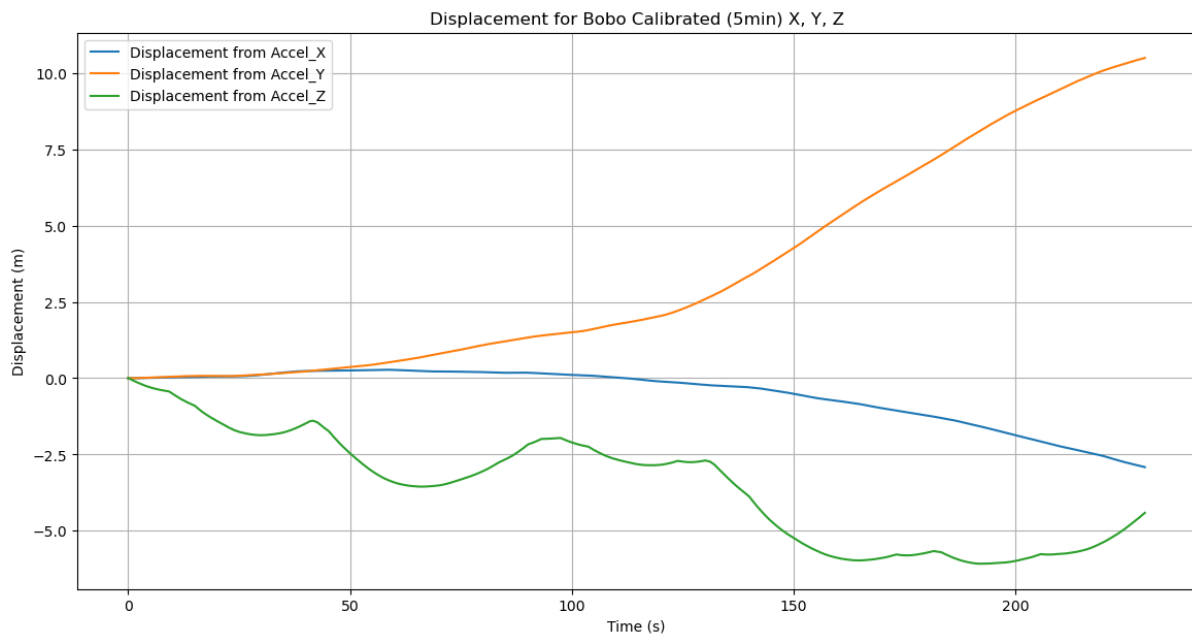
10 HR Calibrated Static Measurement



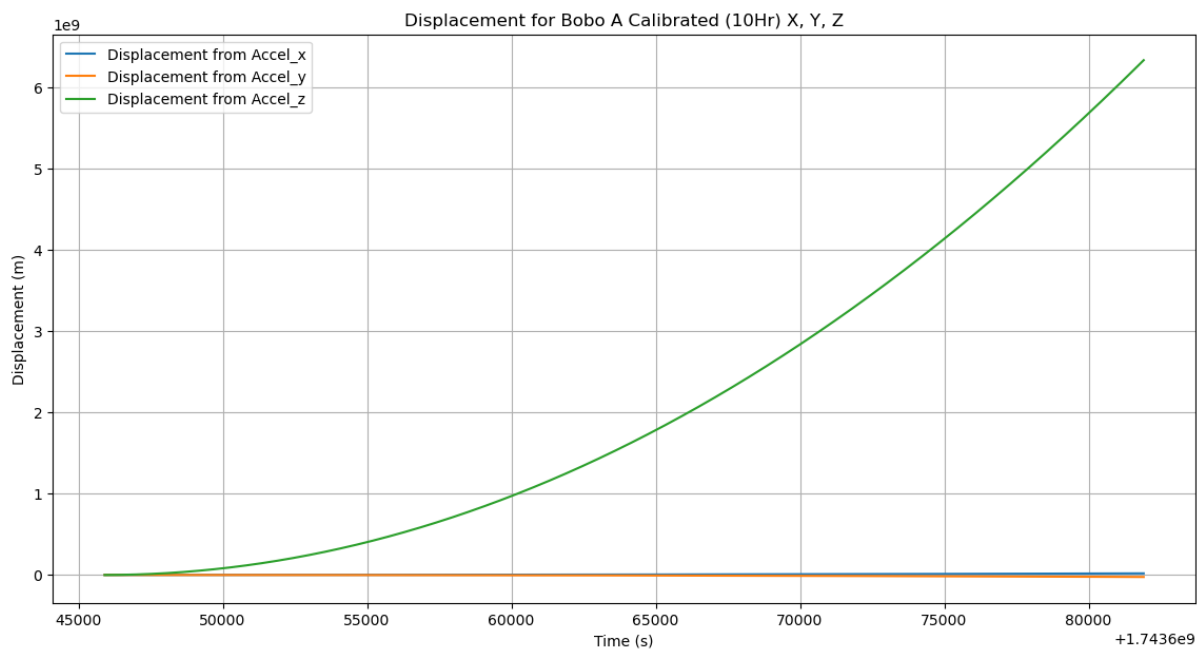
Displacement Comparison Calibrated 1 Min



Calibrated 5 Min



Calibrated 10 HR



Filtered and Unfiltered

[This section was written by Sloan Louden-Robins]

This section highlights how different filtering approaches affect the stability and usability of IMU data, particularly under static conditions. The BNO055 outputs relatively noisy raw accelerometer, gyroscope, and magnetometer data, and this noise appears even when the sensor is not moving. As expected, uncalibrated raw data shows significant offset, bias, and fluctuations that do not reflect true motion.

After calibration, most axes become significantly more stable, especially the accelerometer and gyroscope channels. However, filtering is still needed to extract meaningful long-term trends. Basic smoothing (EMA), windowed median filters, and sensor-fusion-based orientation help reduce random jitter, but none are enough to counteract accumulated drift.

Even though filtering can clean up noise, it does not eliminate the long-term exponential drift caused by IMU integration. This is ultimately why the IMU cannot serve as a primary displacement sensor. Filtering simply makes the short-term data more interpretable—not more accurate over time.

Long-Term

[This section was written by Sloan Louden-Robins]

Long-term testing clearly revealed the main limitation of IMUs for mm-scale structural monitoring: even when perfectly static, the sensor's internal biases accumulate and gradually alter the estimated displacement. Across 5-minute, 1-minute, and 10-hour filtered tests, displacement calculated from integrated acceleration consistently drifted in one direction. This drift is not caused by movement—it is caused by tiny accelerometer offsets being compounded over thousands of samples.

The magnetometer also showed significant variation over long durations due to environmental interference, making heading estimation unreliable in real homes. Even the BNO055's onboard fusion cannot correct for these gradual deviations.

While the IMU performed well as a secondary, context-only sensor (capturing tilt direction, quick changes, or short bursts of motion), it fails completely as a long-term displacement reference. Over multi-hour windows, drift dominates the signal, and no filtering approach—including Kalman—was able to fully correct it.

ToF

[This section was written by Sloan Louden-Robins]

A ToF sensor measures distance by emitting a short burst of light and calculating how long it takes for the reflection to return. This produces highly precise, short-range distance readings that are robust to lighting conditions and surface texture. ToF technology is commonly used in smartphones for facial recognition, robotics for object avoidance, drones, industrial automation, and level-sensing systems. Its advantage is fast, repeatable millimeter-scale distance measurement, making it ideal for displacement tracking. The main trade-offs are sensitivity to target angle, reflective surfaces, and occasional quantization noise in static positions.[4][5][6]

VL53L4CX

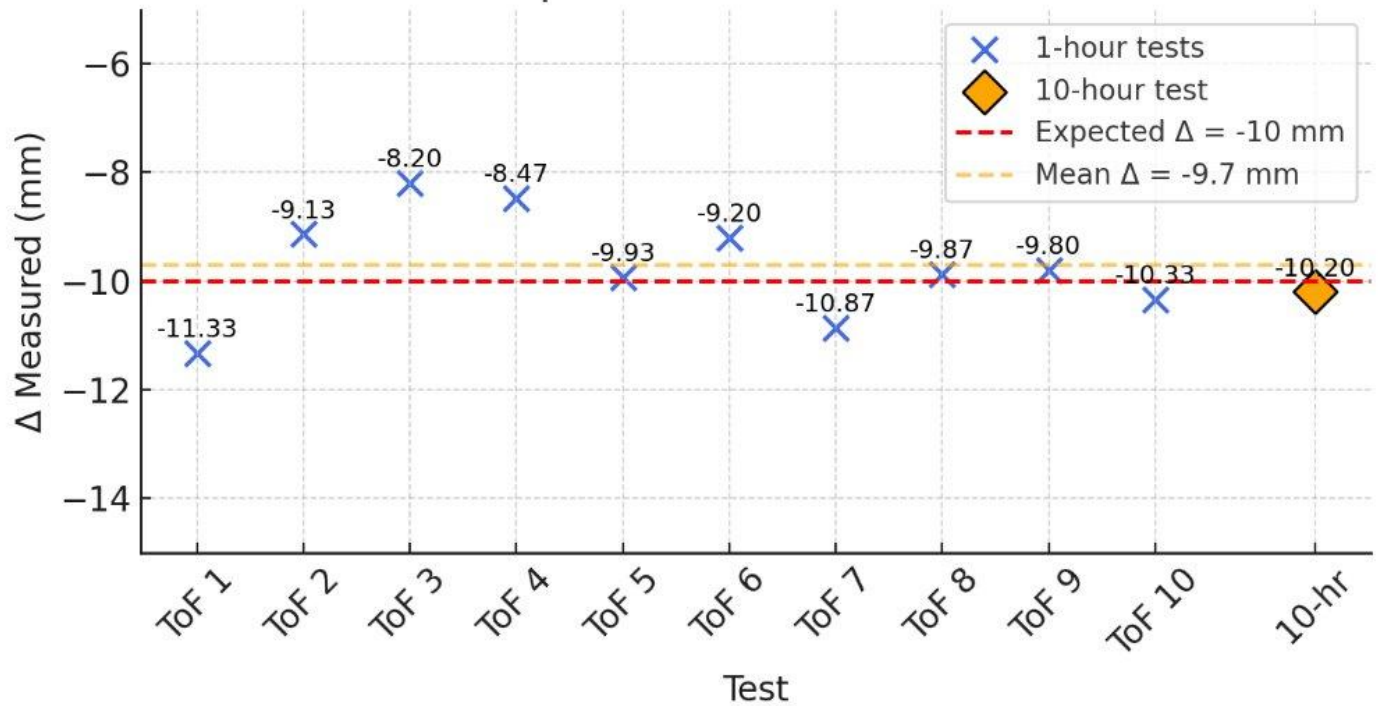
[This section was written by Sloan Louden-Robins and Cassidy Miskovitz]

Datasheet: [Datasheet - VL53L4CX - Time-of-Flight sensor with extended range measurement](#)

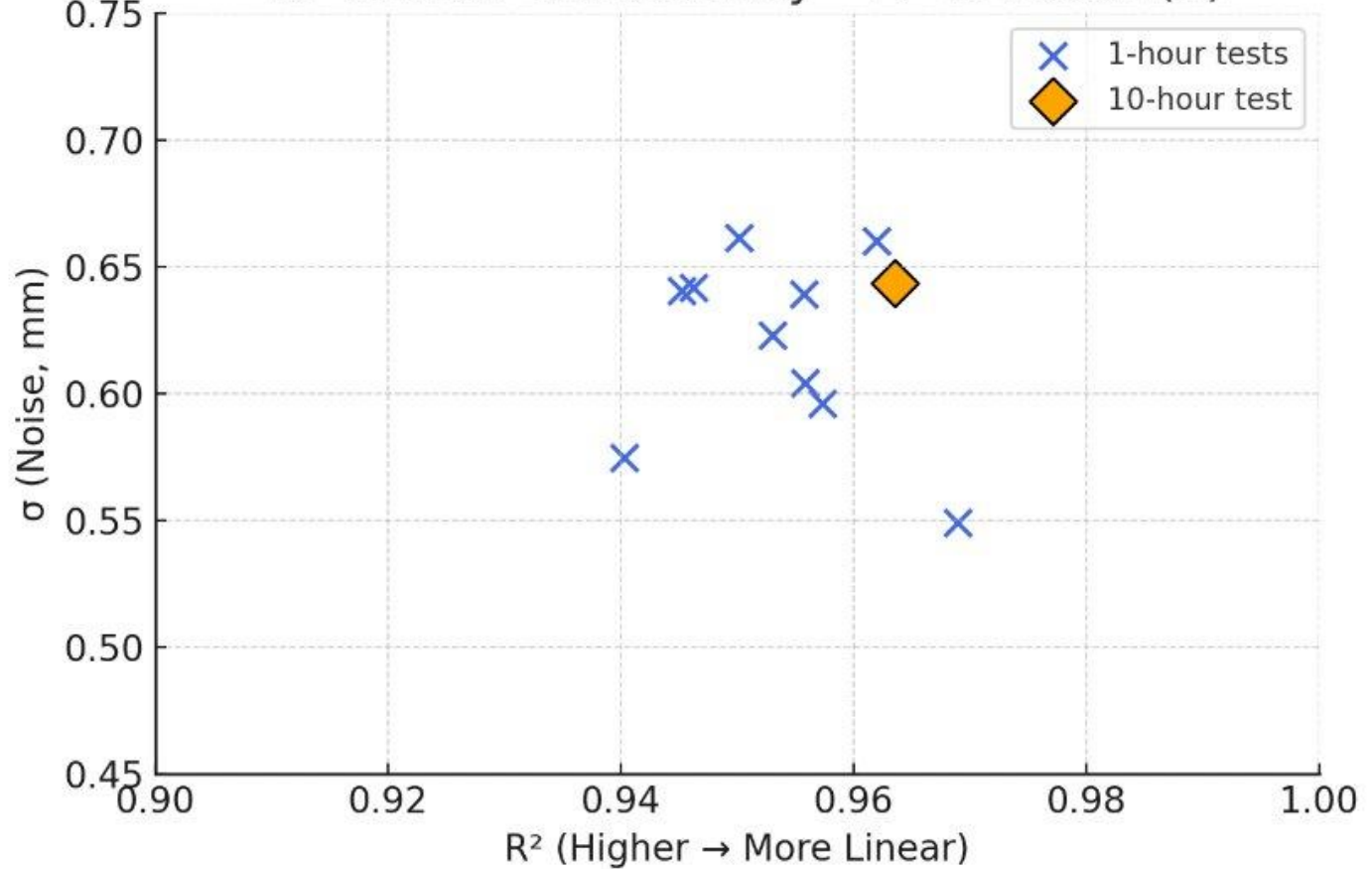
The VL53L4CX is a long-range, high-resolution Time-of-Flight (ToF) sensor that measures distance by timing reflected infrared light. It can detect multiple return objects and provides accurate mm-scale distance readings up to several meters. It is used in robotics, drones, presence detection, and level-sensing applications where precise short-range distance data is needed regardless of ambient lighting.

Observation	Result	Interpretation
1-Hour Tests (n = 10)	$R^2 > 0.94$ $\sigma \approx 0.6 \text{ mm}$	Tight cluster → consistent short-term precision
10-Hour Test	$R^2 = 0.963$ $\sigma = 0.64 \text{ mm}$	Overlaps 1-hr cluster → confirms long-term stability
Outliers / Drift Spread	None observed	All points remain within narrow band → no instability detected
Ideal Reference	$R^2 = 1.0$ $\sigma = 0 \text{ mm}$	perfect linear, zero-noise condition

ToF Measured Displacement - 1 Hour vs 10 Hour Tests



ToF Sensor Consistency - R^2 vs Noise (σ)



Differential pressure

[This section was written by Sloan Louden-Robins]

A differential pressure sensor measures the difference in pressure between two points. This is essential in HVAC systems, fluid-level measurement, aerodynamics, industrial process monitoring, and environmental sensing. When paired with a controlled fluid system (like a water column), differential pressure can be translated into precise height or level differences using known fluid-density relationships. These sensors are fast, stable, and highly sensitive, making them useful for detecting small changes across sealed or semi-sealed systems. [7][8][9]

HSCDRR010NDAA5

[This section was written by Sloan Louden-Robins]

Datasheet: [Datasheet for Honeywell HSCDRR010NDAA5](#)

The HSCDRR010NDAA5 is a high-accuracy, temperature-compensated differential pressure sensor from Honeywell's TruStability line. It measures small pressure differences (± 10 inH₂O range) with a stable analog output and minimal drift. This sensor is used in fluid-level sensing, HVAC systems, air-flow monitoring, environmental systems, and laboratory instrumentation where precise pressure or level differences must be detected over time.

The results for this sensor weren't very thorough, as we received them too late for integration. However, through using a less accurate and less costly differential pressure sensor we found that there is potential with this sensor in the liquid level configuration.

It is recommended to explore this sensor going forward with possible integration in the overall system.

PCR Radar

[This section was written by Sloan Louden-Robins]

Pulsed coherent radar uses radio waves—rather than light—to measure distance, motion, or velocity. By transmitting high-frequency pulses and analyzing the returned signal phase, radar can detect extremely small displacements with high precision, even through dust, fog, or other visual obstructions. PCR is common in automotive ADAS systems, presence detection, industrial monitoring, structural-health sensing, and scientific instrumentation. Unlike optical sensors, radar is less sensitive to lighting or environmental noise, making it suitable for long-term monitoring. It also provides sub-millimeter resolution at close ranges, which makes it a strong candidate for structural-shift analysis and anomaly detection. [10][11][12]

XM125 – A121 Breakout Board

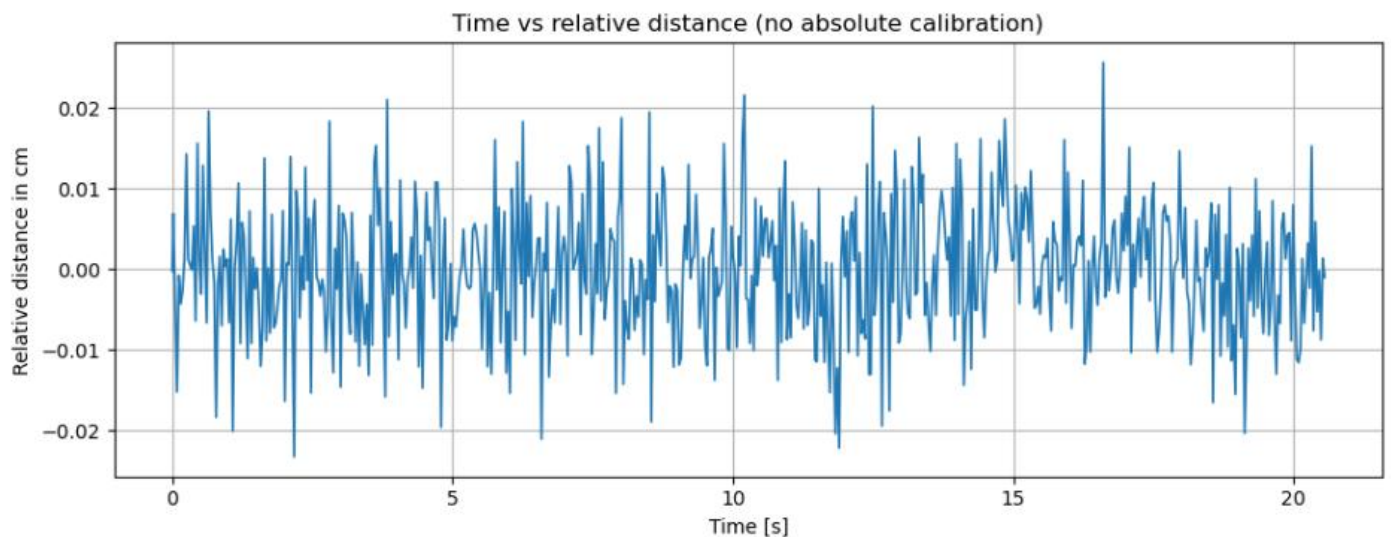
[This section was written by Sloan Louden-Robins]

Datasheet: [A121 Datasheet](#)

The XM125 is a close-range 60-GHz pulsed coherent radar module built around the Acconeer A121 chip. It measures extremely small displacement changes using phase-coherent radar sweeps and can track mm-level movement even through dust, fog, or poor lighting conditions. It is commonly used for

structural monitoring, presence detection, industrial sensing, and any application requiring vibration, displacement, or distance sensing with high robustness to environmental noise.

Overall this sensor became one of the top choices for accuracy and precision in the sub 10mm scale. We specifically explored varying parameters and how that impacted accuracy with a planar reflector. Something interesting we discovered about this sensor is even though light is not supposed to have much impact there was a noticeable difference in accuracy when the reflective panel was placed facing towards the window vs when the sensor was placed towards the sunlight and the panel in the shadow. This may have nothing to do with the light itself but in the reflectivity of the panel. There is also a lot to be discovered in experimenting with optics – specifically lensing the radar sensor, and trying a multitude of panel shapes. More specifically, when we placed a double convex lens in front of the radar, there was an increase in accuracy and precision in static readings. However, due to the time constraints we were not able to further explore this. Similarly, when it comes to the reflective panel's shape and size we were only able to explore a planar surface. After more research it became evident that this system is usually employed with a trihedral design for the panel and the angular reflections could serve to increase overall performance. In the reference material of this document you can see some of the design ideas for the trihedral and some of the ones Orlando 3D printed for testing.



Parameter Settings 0: [Taken from testing log]

Parameter Settings:

```
# Configure detector
detector_config = DetectorConfig(
    start_m=0.020,
    end_m=.5,
    max_profile=a121.Profile.PROFILE_5,
    max_step_length=12,
    threshold_method=ThresholdMethod.CFAR,
    threshold_sensitivity=0.55,
    num_frames_in_recorded_threshold=50,
    close_range_leakage_cancellation=True, # correct field name
    peaksorting_method=PeakSortingMethod.STRONGEST, # notice: no underscore
    reflector_shape=ReflectorShape.PLANAR,
    # [<ReflectorShape.GENERIC: 0>, <ReflectorShape.PLANAR: 1>, <ReflectorShape.SPHERICAL: 2>]
    signal_quality=15.0,
```

Run 1: 51cm static

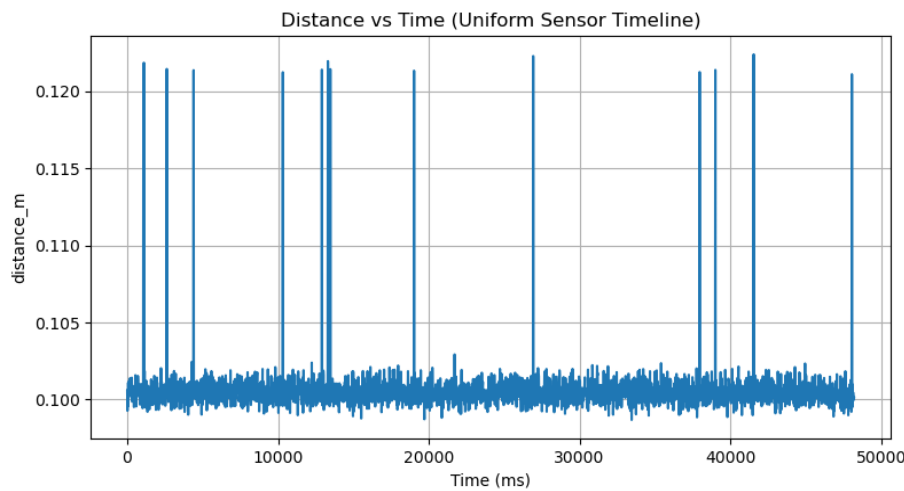


Figure 6. Raw distance vs time plot.

Median Δt = 16.00 ms | Window = 31 samples | σ threshold = 3.0

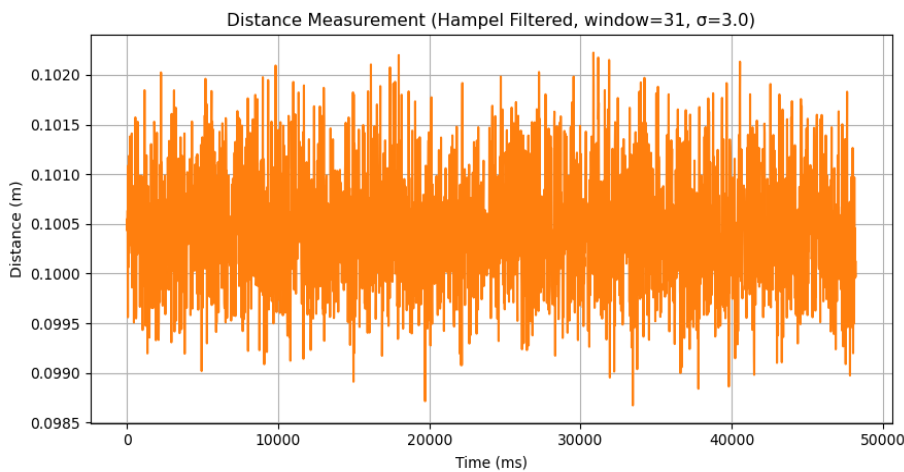


Figure 7. Hampel smoothing filter applied to data, note the image may look noisier but the range in distance has decreased and the window removed the large spikes at 0.1200.

Median Δt = 16.00 ms | EMA α = 0.0363 (half-life $\approx$ 300.0 ms)

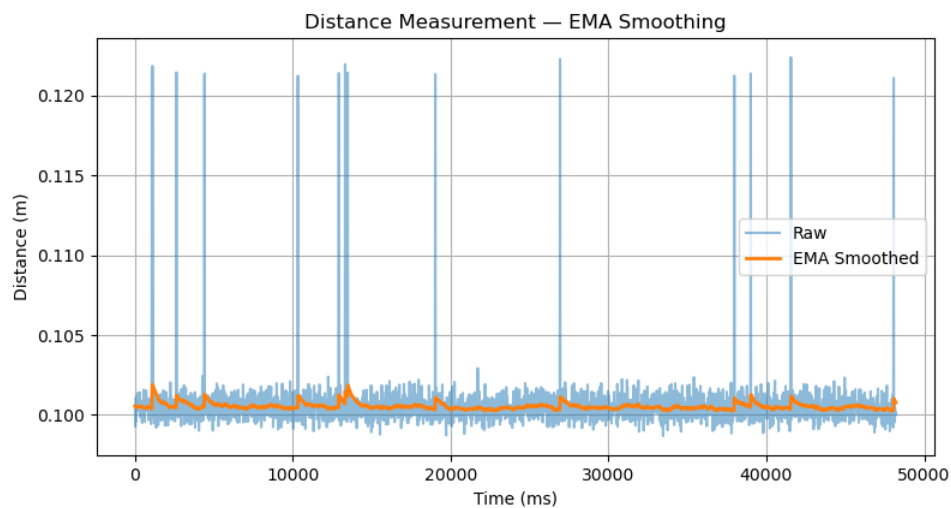


Figure 8. Comparison between EMA smoothed and raw data.

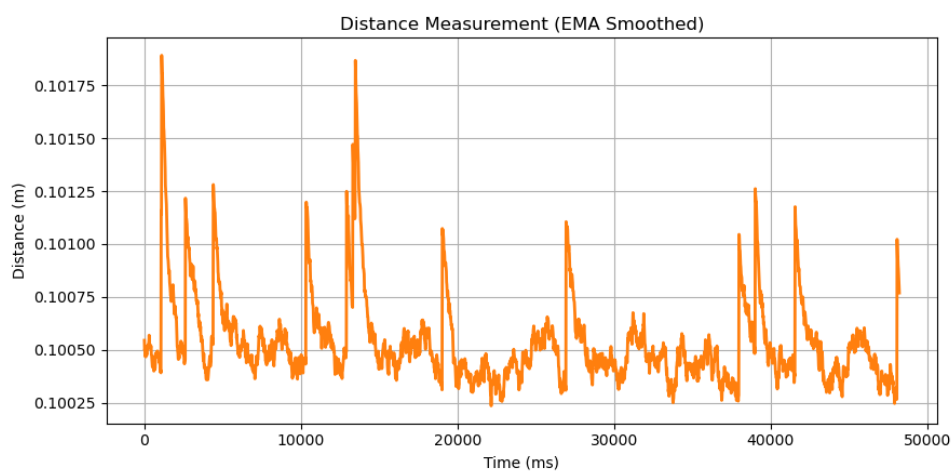


Figure 9. Close up of EMA smoothed plot.

Median Δt = 16.00 ms | Hampel window = 31 | EMA α = 0.0363 | Half-life = 300ms

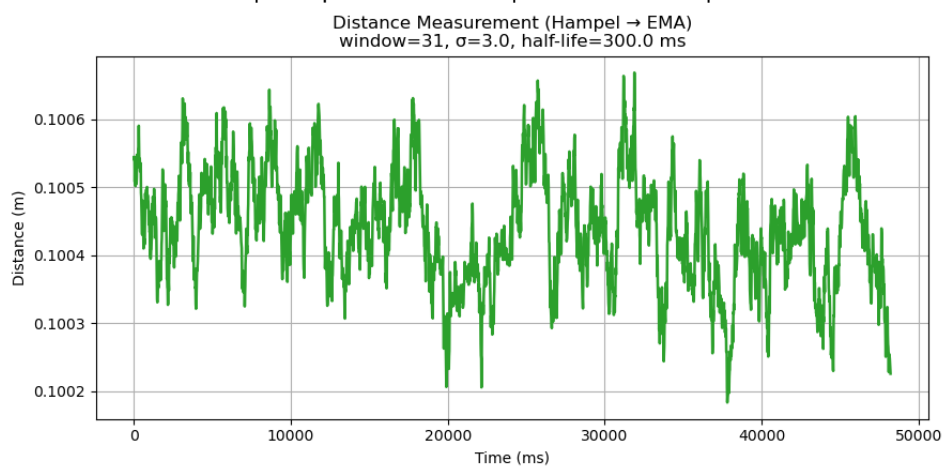


Figure 10. Hampel filter applied to data output before EMA smoothing output.

Median $\Delta t \approx 16.00$ ms | $R=6.719\text{e-}06$ | $Q=3.360\text{e-}07$

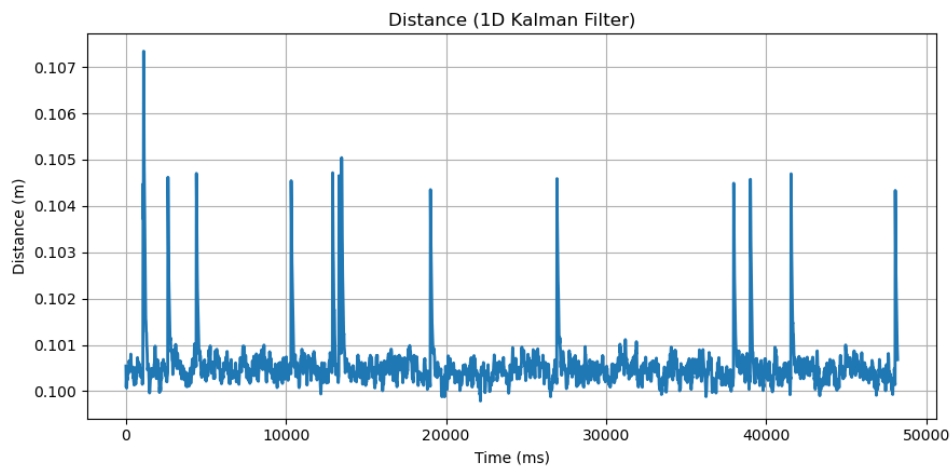


Figure 11. 1D Kalman filter applied

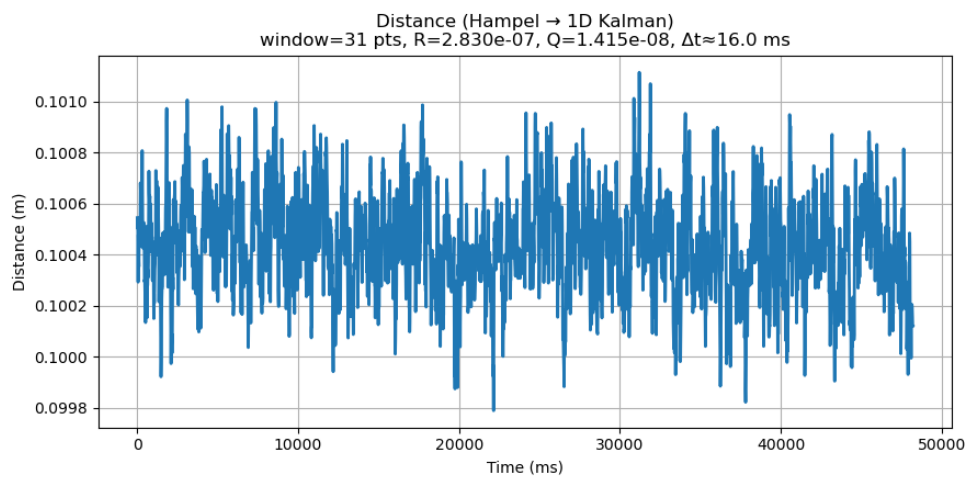


Figure 12. Hampel filter applied before Kalman filter to data

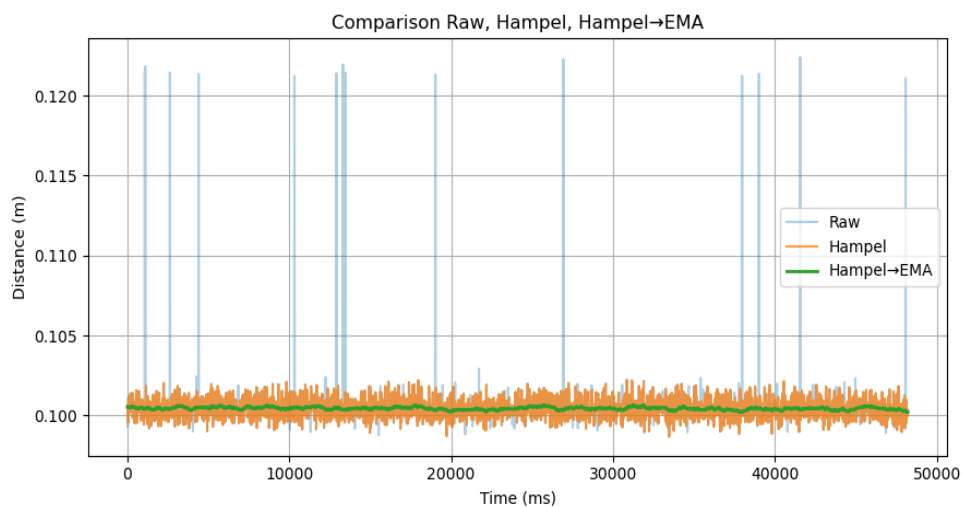


Figure 13. Comparison plot of Raw, Hampel, and Hampel $\rightarrow$ EMA.

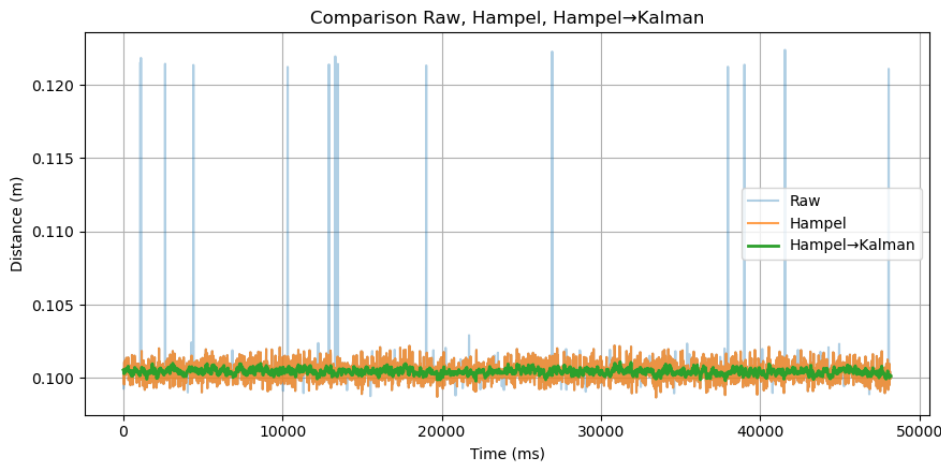


Figure 14. Comparison plot of Raw, Hampel, Hampel -> Kalman.

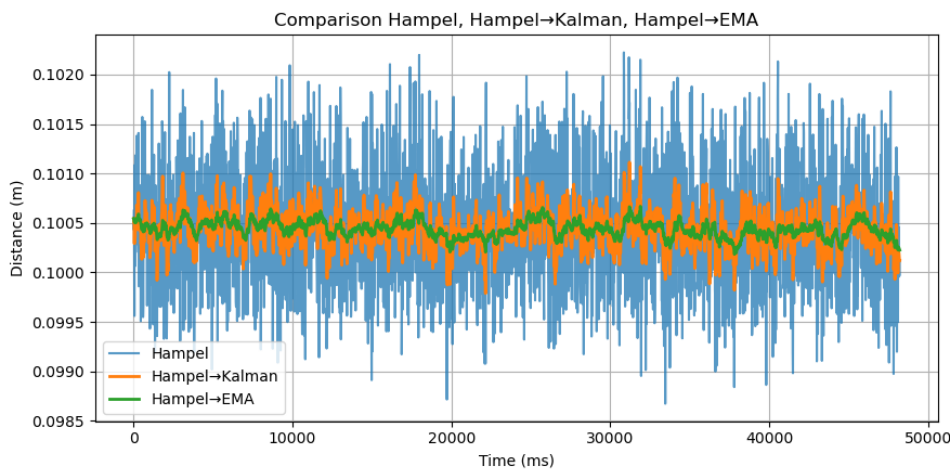


Figure 15. Comparison plot of Hampel, Hampel -> Kalman, Hampel -> EMA.

Results:

Something of note is the distance reading is almost double the actual distance of the reflector. This is likely due to the radar reading the signal there and back where the signal reflecting back off the panel might be experiencing a slight degradation in strength resulting in slightly less than double distance reading. Keeping this in mind it becomes apparent that even with basic filtering techniques the readings will not be accurate unless a threshold is developed in which we half the reading when the panel is placed too close to the radar. This would be implemented in the code on the node relaying this data and be determined during calibration of the system. This study was conducted with a singular panel, so it is undetermined if this placement has an impact on multi-detection settings with many panels in the array. These are factors that will likely impact on our filtering techniques as the system develops. However, in a static, singular object detection, with the parameter configuration 0 reading of a planar panel placed 51mm in distance, the optimal filtering technique intuitively is the Hampel filter applied to before the EMA smoothing function. There could be issues with how precise this model is, meaning we could be overfitting the data and not creating a filter that accurately fits the data.

Dial Gauge

[This section was written by Sloan Louden-Robins]

A digital dial indicator is a precision instrument used to measure very small linear displacements, typically with 1–10 μm resolution. It converts the motion of a spring-loaded plunger into a digital reading using an optical encoder, capacitive sensor, or inductive (LVDT-like) mechanism. These devices are widely used in machining, metrology, calibration labs, and mechanical alignment because they provide stable, low-noise, and highly repeatable measurements. Digital dial indicators are often used as ground-truth displacement references, since they are less sensitive to environmental conditions than IMUs, ToF sensors, or radar.[13][14][15][16]

Although we could not fully integrate the digital dial gauge into our live data pipeline due to time constraints, it remains one of the most promising sensors for future exploration. Its inherent precision and long-term stability make it uniquely valuable for mm-scale displacement verification. Unlike optical or radar-based sensors, the dial gauge provides a direct mechanical measurement that does not rely on signal interpretation, filtering, or environmental compensation. This gives it a level of trustworthiness that is ideal for benchmarking other sensors and establishing a “gold standard” against which IMU, ToF, or radar behavior can be compared.

The main barrier was mechanical integration. The instrument requires a rigid fixture, proper preload, and a consistent point of contact with the moving surface. Designing and fabricating a mount that holds the plunger in the correct alignment—while still allowing the foundation or test platform to move freely—is time-consuming and requires iterative prototyping. Additionally, the gauge’s mechanical components are not designed for outdoor or long-duration field installation without protective housings, making it less practical for rapid prototyping in a student project timeline. In short, the sensor itself performed well, but integrating it into a real-time ESP32-based or Pi-based system required more mechanical and electrical development than the schedule allowed.

Even with these challenges, the digital dial gauge remains an important candidate for future teams. Its readings can serve as a calibration reference for validating ToF and radar drift, tuning filtering parameters, and confirming actual displacement in controlled tests. If a future team dedicates time to building an appropriate mounting assembly and interface electronics, the digital gauge could become a powerful tool both for system validation and for understanding the true performance limits of the other sensors in the suite.

Strain Gauge & Flex Resistor

[This section was written by Sloan Louden-Robins]

Flex Resistor

Datasheet: [SPECTRAFLEX DATA SHEET REV 0.1](#)

A SpectraFlex flex resistor changes resistance as it bends, enabling it to capture curvature or deformation of the surface. These sensors are lightweight, inexpensive, and easy to integrate, which is why they are widely used in wearable devices, motion-capture systems, biomedical tools, and robotics. They are effective for detecting relative movement but have limitations such as non-linear output, temperature sensitivity, and gradual drift, so calibration or compensation is often required when precise measurements are needed.[17][18][19]

Flex resistors are designed for relative motion tracking, not absolute displacement. Their limitations become significant in structural-health applications:

Limitations

- Non-linear response curve → small bends cause inconsistent resistance change.
- Temperature sensitive → ambient heat, HVAC cycling, or sunlight can alter readings independently of foundation movement.
- Drift over time → resistance gradually changes even without motion, making long-term baselines unreliable.
- No absolute reference → it cannot measure actual displacement (mm); it only knows that “bending changed,” not by how much.

Conclusion

Flex sensors are great for gesture detection, robotics, and wearables but cannot provide stable or calibrated mm-scale displacement data. They fail both the long-term stability and accuracy requirements of this project.

Strain Gauge

Datasheet: [2010-10-26-DataSheet-FSR406-Layout2.pdf](#)

A strain gauge measures microscopic deformation (strain) of an object by tracking changes in electrical resistance as the material stretches or compresses. They are foundational in civil engineering, load cells, bridges, structural monitoring, industrial equipment, and research instrumentation. The FSR406 is a pressure/force sensor whose resistance decreases as more force is applied to its surface. It provides qualitative or semi-quantitative force readings and is widely used in human-interface devices, robotics touch sensing, wearable pressure monitoring, and prototyping. FSRs are not precision sensors, but they are useful for detecting contact, pressure changes, and relative force levels.[20][21][22]

While strain gauges are heavily used in structural engineering, the specific type we evaluated (FSR-style force/pressure-based sensors) does not provide the precision needed here.

Limitations

- Not a true strain gauge → FSR406 is a force sensor, not a bonded foil gauge, and is not designed for micro-strain measurements.
- Highly non-linear force–resistance relationship makes calibration challenging.
- Surface-pressure-based, meaning changes in mounting pressure, humidity, or orientation alter results.
- Noise and hysteresis → readings depend on how force is applied and removed, which breaks consistency.
- No fixed reference frame → cannot translate force into actual displacement without a mechanical model.

Conclusion

Because this project requires direct displacement sensing at the mm-scale, a pressure-based force sensor cannot meet requirements. A true bonded foil strain gauge would require custom adhesives, a Wheatstone bridge, environmental compensation, and mechanical coupling to the foundation — far outside project scope.

Soil Moisture

[This section was written by Sloan Loudon-Robins]

A soil-moisture sensor measures the volumetric water content of soil by tracking electrical conductivity or capacitance changes. These sensors are widely used in agriculture, environmental monitoring, irrigation systems, and geotechnical applications. Soil moisture data is valuable for understanding environmental effects on structures, since water content can influence soil expansion, contraction, and overall foundation stability. While not used to measure displacement directly, soil-moisture readings provide important contextual information for long-term structural health analysis.[23][24][25]

Soil moisture plays an important role in foundation movement, but it does not measure displacement and cannot be used as a primary or secondary sensing channel.

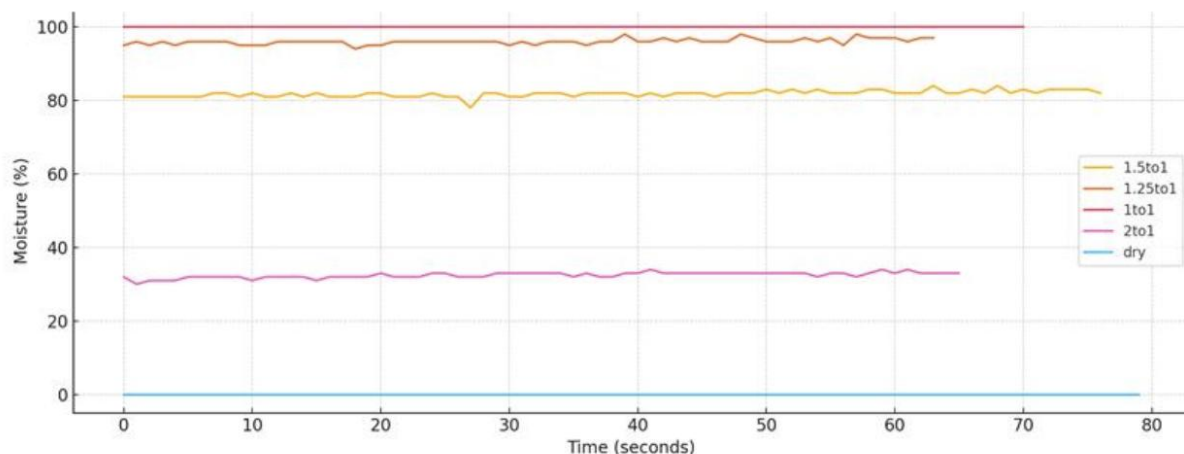
Limitations

- Not a displacement sensor → measures volumetric water content, not linear movement.
- Environmental variability → rainfall, irrigation, groundwater, temperature, and soil composition introduce massive natural fluctuations.
- Probe degradation → soil sensors corrode or drift over months, requiring regular recalibration or replacement.
- Geotechnical variability → soil moisture changes do not map directly to foundation movement without a full soil model and borehole data.

Conclusion

Although valuable as a contextual sensor, soil moisture does not provide meaningful structural measurements. It cannot detect 1–10 mm shifts and therefore cannot be included in the final sensing suite.

COMPARISON OF SOIL MOISTURE OVER TIME ACROSS SAMPLES



Sensor Summary (Cost Benefit Analysis)

[This section was written by Sloan Louden-Robins and Cassidy Miskovitz]

Top 3 Sensor Cost Benefit Analysis:

[This section was written by Cassidy Miskovitz]

How it is calculated:

Unified Equation: Quantifies overall performance across cost–benefit categories:

- **Cost:** lower range → \$10 (best), upper → \$100 (worst)
- **Power:** lower range → 1 mA (best), upper → 100 mA (worst)
- **Accuracy:** 0 % → 100 % (best), 10 % → 0 % (worst) (linear)
- **Precision:** 0.5 mm → 100 % (best), 2.0 mm → 0 % (worst)

$$\text{Score (\%)} = 100 \times \left[1 - \frac{|x_{\text{measured}} - x_{\text{best}}|}{|x_{\text{worst}} - x_{\text{best}}|} \right]$$

- x_measured: Actual value (cost, current, error, or sigma).
- x_best: Ideal or target value (best case).
- x_worst: Upper limit or poorest acceptable value.

Overall Composite Equation: Weighted average of all category scores

$$\text{Overall Composite} = \frac{1}{N} \sum_{i=1}^N \text{Score}_i$$

- Score_i: Individual category score (Cost, Power,Accuracy, Precision, etc.)
- N: Total number of categories included

Category	IMU Measured	Radar Measured	TOF Measured	IMU Score (%)	Radar Score (%)	TOF Score (%)
Hardware Cost	\$30.00	\$50.00	\$15.00	77.8%	55.6%	94.4%
Power Efficiency	12 mA	< 1 mA	25 mA	88.9%	100%	75.8%
Accuracy (Δ error %)	46%	0.7%	2.00 %	0.0%	93.0%	80.0%
Precision (σ)	0.10 °	0.5mm	0.60 mm	—‡	100%	93.3%
Composite Score (avg)				55.6%	87.2%	85.9%

NOTE: IMU precision score is left blank because its measurement is angular (°) rather than linear (mm). Converting angular precision to an equivalent displacement requires defining a lever arm length (R), which varies depending on test geometry and therefore cannot be standardized for this comparison.

Overall Basics Comparison

[This section was written by Sloan Louden-Robins]

Cost = approximate off-the-shelf cost

Computational Load = relative burden on MCU

Lifetime = stability + drift tendency

Performance = suitability for mm-scale foundation monitoring

Sensor	Cost	Computational Load	Lifetime	Performance
MPU6050	\$5-\$12	Medium	Poor	FAIL Can't achieve mm-scale stability
LSM9DS1	\$15-\$25	High	Moderate	FAIL Not viable for long-term displacement
BNO055	\$30-\$45	Low	Poor	FAIL Possible secondary sensor unit
VL53L4CX	\$20	Very low	Excellent	PASS mm-scale accuracy and precision
Differential Pressure	\$45-\$60	Verly low	High	Promising Requires more testing and integration
XM125	\$60-\$90	Moderate	Excellent	PASS Strongest precision after ToF
Dial Gauge	\$45	Very low	Excellent	Promising Requires more exploration – heavy mechanical installation
SpectraFlex	\$7-\$12	Very low	Poor	FAIL Non-linear, not accurate enough
Strain Gauge	\$5-\$10	Low	Poor	FAIL Measuring force proportional to displacement isn't possible with how heavy houses are
Soil Moisture	\$10-\$25	Very Low	Moderate	FAIL Not applicable for displacement, context only sensor

The table compares all major sensors explored in this project IMU, ToF, radar, dial gauge, strain, flex, soil moisture, and differential pressure, based on cost, computational load, lifetime, and performance.

In summary:

- **IMUs** are inexpensive and computationally light but fail long-term stability tests.
- **ToF** sensors offer excellent precision at low cost and low computational overhead, making them one of the strongest candidates for real deployment.
- **Radar** (XM125/A121) provides the highest precision and environmental robustness but comes with higher cost and greater integration difficulty.
- **Dial gauges** are extremely precise but impractical for real homes due to the complexity of mechanical installation.
- **Strain/flex** sensors are low-cost and lightweight, but not precise enough for mm-scale structural monitoring.
- **Differential pressure** sensors show promise in hydrostatic configurations but require more validation.
- **Soil moisture** sensors are not displacement sensors but are valuable as contextual monitors for environmental conditions.

Overall, ToF and radar emerged as the best long-term fits for the system's required accuracy and reliability.

Filtering and Machine Learning

[This section was written by Sloan Louden-Robins]

Overview

Filtering and machine learning played a central role in making the sensor data usable, stable, and meaningful for long-term structural monitoring. Because each sensor type produces different noise patterns, drift behaviors, and sampling characteristics, the project required a modular processing pipeline capable of cleaning and interpreting data in a consistent way.

Filters

[This section was written by Sloan Louden-Robins]

Overview

Filtering was one of the most important portions of the data-processing pipeline. Each sensor behaved differently, so a single universal filter was not possible. Instead, a modular filtering suite was developed to support IMU, ToF, and radar datasets. These filters were used for noise rejection, outlier removal, timestamp smoothing, and long-duration stability analysis.

What's Been Achieved

- Implemented a complete filtering toolkit inside Jupyter:
- Hampel outlier rejection
- Exponential Moving Average (variable half-life)
- Butterworth low-pass filter
- Windowed median filters
- Madgwick and Kalman smoothing
- Built automated pipelines applying filters sequentially (e.g., Hampel → EMA)
- Validated filtering consistency between Python and C++ for radar and ToF
- Tuned half-life values and window sizes specifically for each sensor type
- Created comparison plots showing how each filter affects noise and drift

Key Takeaways

- Hampel is the single most reliable first-stage filter: it removes spikes without over smoothing.
- EMA is ideal for continuous displacement signals when tuned to the correct half-life.
- The best-performing pipeline for radar static displacement was:
- Hampel → EMA
- IMU filtering cannot meaningfully correct long-term drift, only improves short-term stability.
- Filtering needs to be tuned per sensor type; no single universal configuration works across all sensors.

Future Suggestions

- Add automated parameter sweeps to identify the optimal EMA half-life and Hampel thresholds.
- Introduce frequency-domain filtering for radar and IMU spectral analysis.
- Bundle the filtering toolkit as a Python package for the next team.
- Add automated filter validation tests so new sensors can be integrated quickly.
- Identify noise type by taking FFT of signal – this could prove insightful for how to remove the noise

Machine Learning

[This section was written by Sloan Loudon-Robins]

Overview:

Machine learning was explored as an extension of the filtering pipeline, with a focus on trend modeling, drift detection, and anomaly classification. ML was not meant to replace physical filtering, but rather to identify subtle long-term patterns that are difficult to detect through classical methods. The first ML implementations focused on IMU drift modeling and cross-sensor feature correlation.

What's Been Achieved:

- Implemented the first ML pipeline in the project using Python + scikit-learn + TensorFlow.
- Trained an LSTM model on the two-week BNO055 dataset to predict yaw drift.
- Created correlation matrices to identify which IMU features behave consistently in static conditions.
- Built early feature-extraction tools (accel magnitude, gravity vector, orientation deltas).
- Configured the LEAP2 cluster to support large dataset training and long-duration drift modeling.

Key Takeaways:

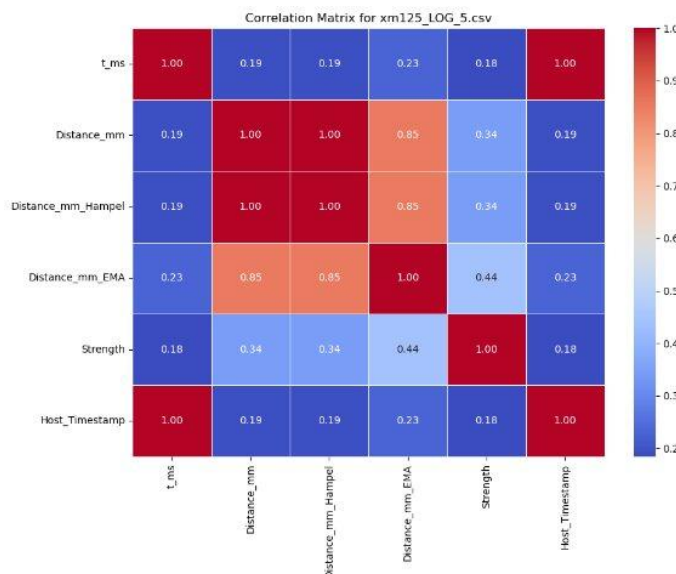
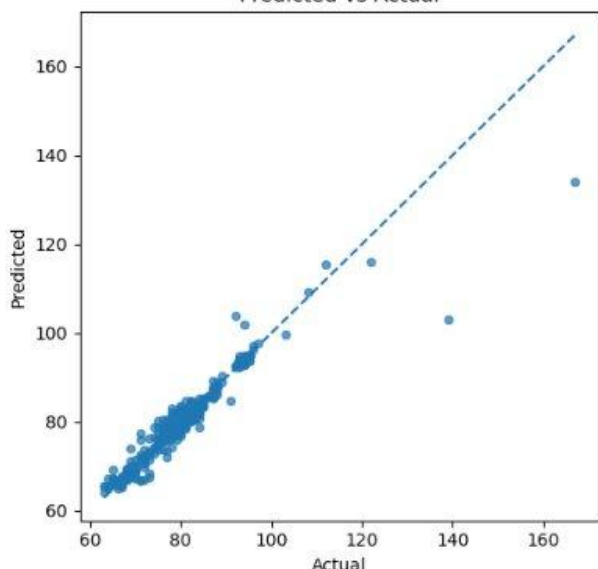
- ML works best when the underlying sensor is stable; IMU drift made long-term prediction trivial (LSTM predicted a constant line).
- Radar and ToF data would benefit most from ML in the future, especially for shift detection and classification.
- ML cannot salvage inherently unstable sensors (e.g., IMUs for displacement).
- Proper timestamp alignment and filtering matter more than model complexity.

Future Suggestions:

- Train ML models on ToF + radar combined datasets to identify unique displacement signatures.
- Develop anomaly-detection models that trigger alerts based on deviation from normal structural patterns.
- Build ML Feature Extractors that run automatically (daily or weekly) on the UI side.
- Test classical ML (Random Forest, SVM) against neural approaches for early shift classification.

[Test Metrics] $R^2=0.9467$ | $MAE=0.5195$ | $RMSE=1.7040$

Predicted vs Actual



Detected task: REGRESSION

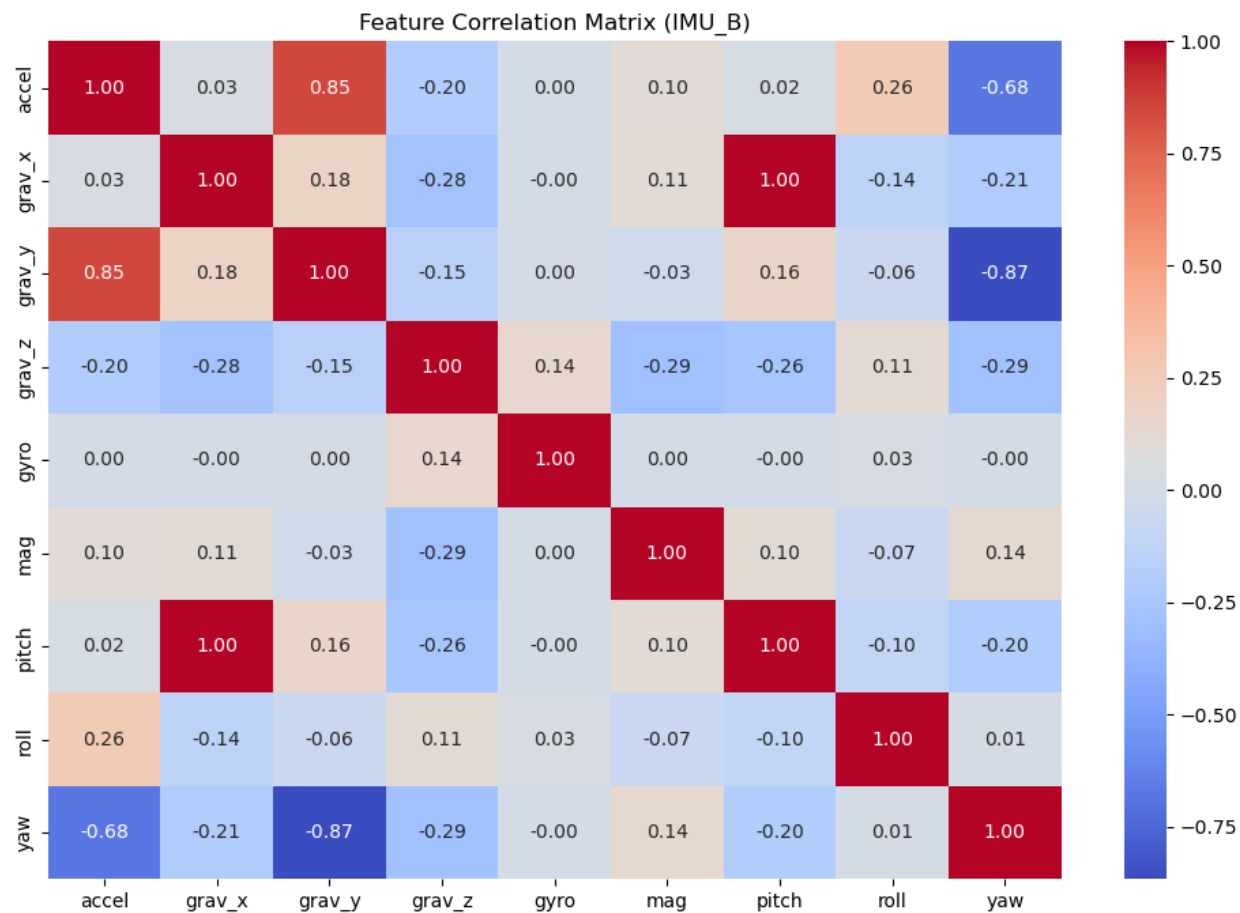
Ridge | CV mean=0.7420 std=0.0670

RandomForest | CV mean=0.9643 std=0.0055

KNN | CV mean=0.9194 std=0.0221

IMU – BNO055 – 2wk static drift ML model and analysis

Correlation Matrix



Interpretation:

The correlation analysis of the BNO055 two-week static dataset confirms that the sensor remained fully stationary throughout the test period. The strongest correlations occur between **accel**, **gravity vector components**, and **yaw**, reflecting fixed geometric relationships caused by the device’s orientation. Specifically, accel magnitude and grav_y show a strong positive correlation (+0.85), while yaw and grav_y show a strong negative correlation (−0.87), indicating the sensor maintained a consistent heading and tilt relative to gravity. Gyroscope and magnetometer magnitudes exhibit near-zero correlation with all other features, which is expected in a no-motion environment and confirms an absence of rotational or magnetic disturbances.

Overall, the correlation matrix validates the stability of the BNO055 under long-duration static conditions and provides a clean baseline for evaluating drift. The relationships observed are dominated by static orientation geometry rather than sensor dynamics, consistent with a sensor experiencing negligible movement or environmental variation.

Output Screenshots

```

Unnamed: 0    result table      _start \
0          NaN  _result        0  1970-01-01T00:00:00Z
1          NaN  _result        0  1970-01-01T00:00:00Z
2          NaN  _result        0  1970-01-01T00:00:00Z
3          NaN  _result        0  1970-01-01T00:00:00Z
4          NaN  _result        0  1970-01-01T00:00:00Z

      _stop      _time \
0  2025-10-23T02:05:19.527412439Z  2025-09-30T02:17:58.522175313Z
1  2025-10-23T02:05:19.527412439Z  2025-09-30T02:17:59.524045687Z
2  2025-10-23T02:05:19.527412439Z  2025-09-30T02:18:00.594213951Z
3  2025-10-23T02:05:19.527412439Z  2025-09-30T02:18:01.655051086Z
4  2025-10-23T02:05:19.527412439Z  2025-09-30T02:18:02.718497139Z

   _measurement host      topic accel  ... grav_y grav_z gyro mag \
0  mqtt_consumer  pi  sensors/imu_B    0  ...    0.0    0.0    0    0
1  mqtt_consumer  pi  sensors/imu_B    0  ...   -0.86    9.76    3    0
2  mqtt_consumer  pi  sensors/imu_B    0  ...   -0.86    9.76    3    0
3  mqtt_consumer  pi  sensors/imu_B    0  ...   -0.86    9.76    3    0
4  mqtt_consumer  pi  sensors/imu_B    0  ...   -0.86    9.76    3    0

   pitch    roll sys temperature ts_ms    yaw
0     0.0     0.0  0             0  5339    0.0
1  -0.3125  359.9375  0             25  6396  5.0625
2  -0.3125  359.9375  0             26  7459  5.0625
3  -0.3125  359.9375  0             26  8517  5.0625
4  -0.3125  359.9375  0             26  9585  5.0625

```

```

[5 rows x 21 columns]
Columns: ['Unnamed: 0', 'result', 'table', '_start', '_stop', '_time', '_measurement', 'host', 'topic', 'accel', 'grav_x', 'grav_y', 'grav_z', 'gyro', 'mag', 'pitch', 'roll', 'sys', 'temperature', 'ts_ms', 'yaw']

```

```

(1604870, 21)
X_raw shape: (1602692, 8)
y_raw shape: (1602692,)
Sequence shape: (1602660, 32, 8)
Labels shape: (1602660,)
Train: (1282128, 32, 8)
Test : (320532, 32, 8)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 32, 64)	18,688
lstm_1 (LSTM)	(None, 32)	12,416
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

Total params: 31,649 (123.63 KB)

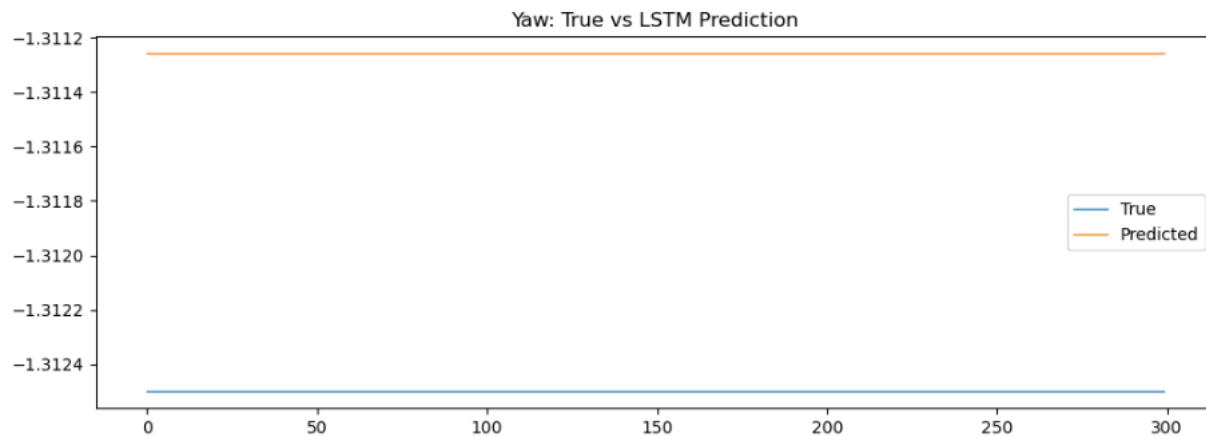
Trainable params: 31,649 (123.63 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/25
16027/16027 ————— 594s 37ms/step - loss: 0.9566 - mae: 0.0360 - val_loss: 0.0024 - val_mae: 0.0398
Epoch 2/25
16027/16027 ————— 604s 38ms/step - loss: 0.1655 - mae: 0.0227 - val_loss: 0.0015 - val_mae: 0.0350
Epoch 3/25
16027/16027 ————— 604s 38ms/step - loss: 0.1812 - mae: 0.0202 - val_loss: 0.0023 - val_mae: 0.0395
Epoch 4/25
16027/16027 ————— 568s 34ms/step - loss: 0.1395 - mae: 0.0192 - val_loss: 0.0019 - val_mae: 0.0336
Epoch 5/25
16027/16027 ————— 554s 35ms/step - loss: 0.1301 - mae: 0.0184 - val_loss: 0.0020 - val_mae: 0.0353
Epoch 6/25
16027/16027 ————— 569s 35ms/step - loss: 0.1225 - mae: 0.0181 - val_loss: 0.0027 - val_mae: 0.0399
Epoch 7/25
16027/16027 ————— 548s 34ms/step - loss: 0.1120 - mae: 0.0170 - val_loss: 0.0018 - val_mae: 0.0309
Epoch 8/25
16027/16027 ————— 536s 33ms/step - loss: 0.1473 - mae: 0.0175 - val_loss: 0.0020 - val_mae: 0.0358
Epoch 9/25
16027/16027 ————— 544s 34ms/step - loss: 0.0879 - mae: 0.0167 - val_loss: 0.0020 - val_mae: 0.0343
Epoch 10/25
16027/16027 ————— 610s 38ms/step - loss: 0.1177 - mae: 0.0170 - val_loss: 0.0018 - val_mae: 0.0318
Epoch 11/25
16027/16027 ————— 630s 39ms/step - loss: 0.1077 - mae: 0.0170 - val_loss: 0.0019 - val_mae: 0.0350
Epoch 12/25
16027/16027 ————— 688s 40ms/step - loss: 0.0957 - mae: 0.0164 - val_loss: 0.0020 - val_mae: 0.0323
Epoch 13/25
16027/16027 ————— 3055s 188ms/step - loss: 0.0955 - mae: 0.0160 - val_loss: 0.0018 - val_mae: 0.0324
Epoch 14/25
16027/16027 ————— 633s 39ms/step - loss: 0.0800 - mae: 0.0176 - val_loss: 0.0019 - val_mae: 0.0335
Epoch 15/25
16027/16027 ————— 666s 42ms/step - loss: 0.0919 - mae: 0.0166 - val_loss: 0.0025 - val_mae: 0.0389
Epoch 16/25
16027/16027 ————— 608s 38ms/step - loss: 0.0793 - mae: 0.0162 - val_loss: 0.0027 - val_mae: 0.0392
Epoch 17/25
16027/16027 ————— 583s 36ms/step - loss: 0.0772 - mae: 0.0150 - val_loss: 0.0023 - val_mae: 0.0378
Epoch 18/25
16027/16027 ————— 703s 41ms/step - loss: 0.0743 - mae: 0.0155 - val_loss: 0.0019 - val_mae: 0.0317
Epoch 19/25
16027/16027 ————— 617s 38ms/step - loss: 0.0808 - mae: 0.0158 - val_loss: 0.0020 - val_mae: 0.0351
Epoch 20/25
16027/16027 ————— 630s 39ms/step - loss: 0.0672 - mae: 0.0137 - val_loss: 0.0018 - val_mae: 0.0347
Epoch 21/25
16027/16027 ————— 590s 37ms/step - loss: 0.0887 - mae: 0.0158 - val_loss: 0.0020 - val_mae: 0.0318
Epoch 22/25
16027/16027 ————— 537s 34ms/step - loss: 0.0604 - mae: 0.0135 - val_loss: 0.0020 - val_mae: 0.0316
Epoch 23/25
16027/16027 ————— 553s 34ms/step - loss: 0.0547 - mae: 0.0120 - val_loss: 0.0017 - val_mae: 0.0309
Epoch 24/25
16027/16027 ————— 535s 33ms/step - loss: 0.0760 - mae: 0.0131 - val_loss: 0.0016 - val_mae: 0.0342
Epoch 25/25
16027/16027 ————— 538s 34ms/step - loss: 0.0634 - mae: 0.0124 - val_loss: 0.0020 - val_mae: 0.0331
10017/10017 ————— 98s 10ms/step
Test MAE: 0.018
Test RMSE: 0.028

Test MAE = 0.018 rad $\times (180/\pi) \approx 1.03^\circ$

Test RMSE = 0.028 rad $\times (180/\pi) \approx 1.60^\circ$



Interpretation:

During the static 2-week BNO055 test, the yaw signal showed negligible variation, remaining constant within $\sim\pm 1^\circ$. As a result, the LSTM model converged toward predicting the mean yaw value, producing a flat-line prediction with an MAE of 0.018 rad ($\sim 1^\circ$) and RMSE of 0.028 rad ($\sim 1.6^\circ$). This confirms that the sensor exhibited no meaningful drift over this window and that the LSTM learned the correct mapping for a stationary dataset.

Model details:

The model is a two-layer LSTM neural network with 64 and 32 units, followed by two dense layers (16-unit ReLU and a 1-unit linear output). It processes 32-sample IMU sequences containing 8 fused features and predicts a single yaw value. It contains 31,649 trainable parameters and is optimized with Adam using MSE loss over 25 epochs

Parameter	Value
Sequence length (SEQ_LEN)	32 time steps
Features per timestep	8 features
Input tensor shape	(batch_size, 32, 8)

The 8 Features are:

accel

grav_x, grav_y, grav_z

gyro

mag

pitch

roll

Using **Parameter count formula:**

$\text{LSTM params} = 4 \times (\text{units} \times (\text{units} + \text{input_dim} + 1))$

Layer 1 — LSTM	Layer 2 — LSTM
• Type: LSTM • Units: 64 • Return sequences: True • Input shape: (32, 8) • Output shape: (32, 64) Parameters: $= 4 \times (64 \times (64 + 8 + 1))$ $= 4 \times (64 \times 73)$ = 18,688 parameters	• Type: LSTM • Units: 32 • Return sequences: False • Input shape: (32, 64) • Output shape: (32) Parameters: $= 4 \times (32 \times (32 + 64 + 1))$ $= 4 \times (32 \times 97)$ = 12,416 parameters
Layer 3 — Dense	Layer 4 — Dense Output
• Units: 16 • Activation: ReLU • Input: 32 • Output: 16 Parameters: $= 32 \times 16 + 16$ = 528 parameters	• Units: 1 • Activation: Linear (default) • Purpose: Regression output → predicts yaw value Parameters: $= 16 \times 1 + 1$ = 17 parameters

Total Trainable Parameters:

Layer	Params
LSTM (64)	18,688
LSTM (32)	12,416
Dense (16)	528
Dense (1)	17
Total	31,649 trainable parameters

Training Configuration:

Parameter	Setting
Optimizer	Adam
Learning rate	default (0.001)
Loss function	MSE (Mean Squared Error)
Metrics	MAE (Mean Absolute Error)
Epochs	25
Batch size	64
Validation split	0.2 (20% of training data)

Model output shape:

(batch_size, 1)

The model predicts **one yaw value** per 32-sample window.

LSTM Architecture:

The drift-prediction model uses a 2-layer recurrent LSTM network with 64 units in the first layer (returning sequences) and 32 units in the second layer. The recurrent stack is followed by a 16-unit fully connected layer with ReLU activation and a final linear output neuron. The network processes 32-sample windows of 8 fused IMU features (gravity vector, acceleration magnitude, gyroscope magnitude, magnetometer magnitude, pitch, and roll) and outputs a single yaw estimate. The total number of trainable parameters is 31,649. The model was trained using the Adam optimizer with MSE loss, a batch size of 64, and 25 epochs, using 20% of the training data for validation.

Configurations

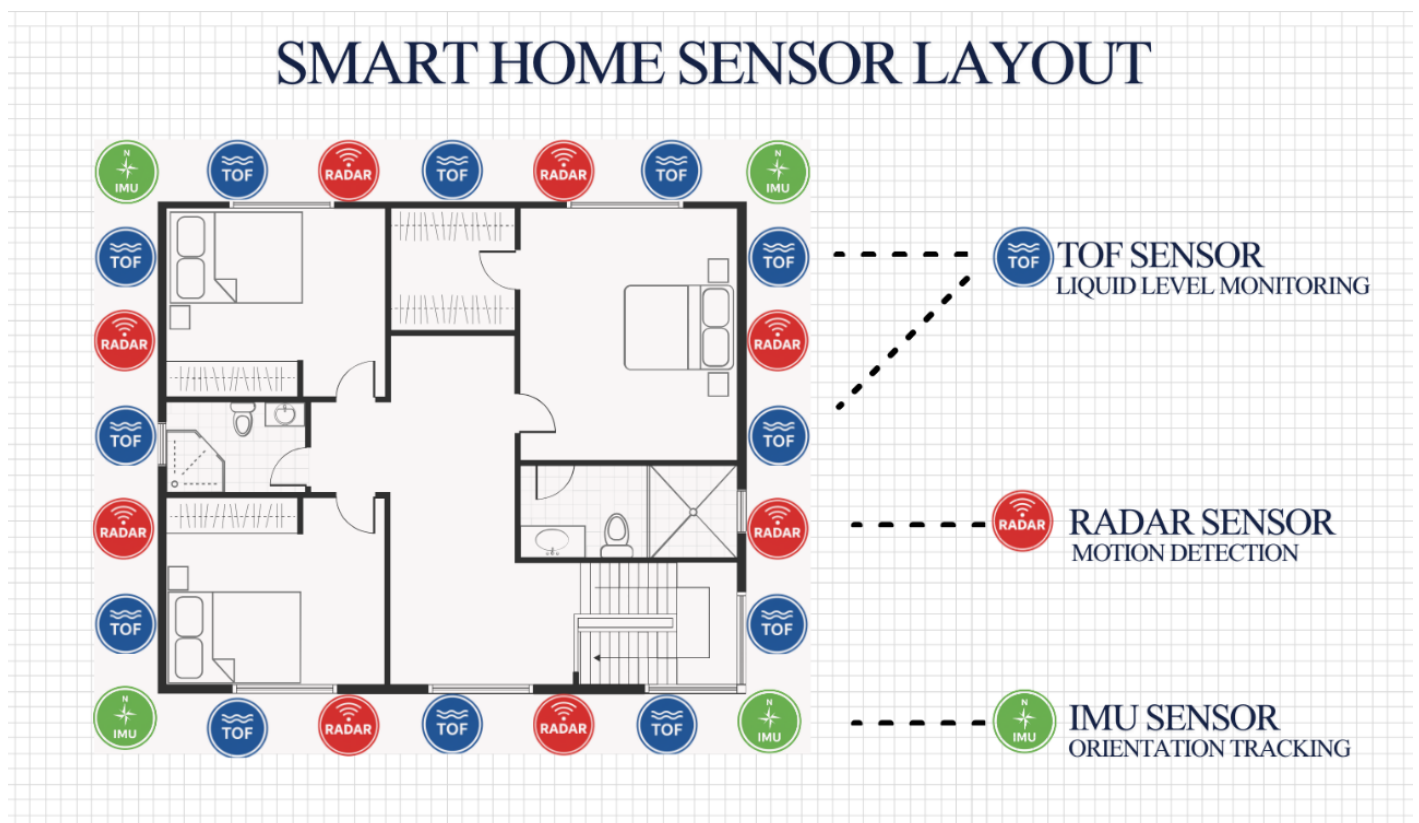
[This section was written by Sloan Louden-Robins]

Overview

There is a highly likely chance that one sensor alone will not be able to achieve the level of precision and accuracy this project is hoping to prove is possible. That's where sensor fusion comes into play and should be explored thoroughly. This is arguably more important than continuing research on individual new sensors. Different configurations of a sensor suite will allow for more flexibility and stability, the trade off comes with complexity. Due to the limited time constraints our team could not implement any sensor configurations with varying sensor types, but Cassidy Miskovitz was able to propose a sensor fusion that could work with our current sensors.

The proposed configurations include the ToF liquid level system and radar modules being placed strategically around the perimeter of the home's foundation, with possible integration of IMU sensors placed at each main corner, functioning as a secondary indicator of structural displacement. Some drawbacks to this system are how the sensors will be housed because they are placed outside of the home exposed to the elements. Another concern is cost, the IMU and radar sensors have a higher off the shelf cost so as these scales on larger homes the cost might outweigh what is reasonable for the average homeowner. In contrast, the level of precision and accuracy demonstrated by the ToF and radar show serious promise being used in conjunction. Even with these trade-offs, the combined ToF + radar configuration is the strongest candidate for achieving <5 mm stability with minimal filtering. This hybrid approach provides redundancy, reduces false positives, and offers stronger long-term confidence than any single-sensor system.

Proposed Fusion by Cassidy Miskovitz:



Citations

- [1] Titterton, David, and John Weston. *Strapdown Inertial Navigation Technology*. IET, 2004.
- [2] Mikolajczyk, T., et al. "Inertial sensors and their applications." *Sensors*, vol. 21, no. 23, 2021.
- [3] Bourke, A. K., & Lyons, G. M. "A threshold-based fall-detection algorithm using inertial sensors." *IEEE Trans. Biomedical Engineering*, 2008.
- [4] Lange, R., & Seitz, P. "Solid-state time-of-flight distance sensor." *IEEE Journal of Quantum Electronics*, 2001.
- [5] Kolodziej, K. E., et al. "A survey of active optical ToF sensors." *Optical Engineering*, vol. 59, no. 6, 2020.
- [6] Opris, I., "Time-of-flight camera principles." *Swiss Federal Institute of Technology (ETH Zurich) Lecture Notes*, 2010.
- [7] Taylor, G. I. "On differential pressure measurements." *Proceedings of the Royal Society A*, 1938.
- [8] Miller, R. W. *Flow Measurement Engineering Handbook*. McGraw-Hill.
- [9] Rahimi, M., et al. "Review of pressure sensors for harsh environments." *Sensors*, 2021.
- [10] Hao, Yunhong, et al. "Millimeter-wave radar for displacement measurement." *IEEE Sensors Journal*, 2020.
- [11] Massagram, W., et al. "A novel noncontact displacement sensor using 60 GHz FMCW radar." *IEEE Transactions on Microwave Theory and Techniques*, 2009.
- [12] Gürbüz, S. Z., & Guldogan, M. B. "Coherent radar sensing: fundamentals." *IEEE Radar Conference Tutorials*, 2019.
- [13] Genta, G. *Mechanical Measurements*. Springer.
- [14] Smith, G. T. *Industrial Metrology*. Springer.
- [15] Ichimiya & Saito. "High-precision displacement measurement using optical encoders." *Precision Engineering*, 1992.
- [16] Fraden, J. *Handbook of Modern Sensors*. Springer.
- [17] Saggio, Giovanni, and Massimo Guglielmelli. "A novel bend sensor for measuring finger flexion." *Sensors and Actuators A: Physical*, 2010.
- [18] Tognetti, A., et al. "Wearable sensing for hand posture and gesture recognition." *IEEE Sensors Journal*, 2013.
- [19] Dipietro, L., et al. "A survey of glove-based systems and their applications." *IEEE Transactions on Systems, Man, and Cybernetics*, 2008.
- [20] Chen, Z., et al. "Strain sensors: review of materials and mechanisms." *Advanced Materials*, 2021.
- [21] Doebelin, E. O. *Measurement Systems: Application and Design*. McGraw-Hill.
- [22] Sohn, H., et al. "A review of structural health monitoring literature." *Los Alamos National Laboratory Report*, 2003.
- [23] Evett, S. R., et al. "Soil water sensing for water balance." *Vadose Zone Journal*, 2008.
- [24] Robinson, D. A., et al. "Soil moisture measurement systems: a review." *Vadose Zone Journal*, 2008.
- [25] Campbell, G. S., *Soil Physics with BASIC*. Elsevier Academic Press.